ZSA Guide

Zcash Shielded Assets: issuance, transfer, and the counterfeiting boundary

m@rek.onl

Abstract

Assuming the Orchard protocol of the *Ironwood Guide*, this volume of *The Zcash Arboretum* documents Zcash Shielded Assets: per-asset value bases and asset identity, transparent issuance and finalization (ZIP-227), shielded transfer and burn (ZIP-226), and the split-note construction that guards the counterfeiting boundary. The upstream ZIPs are in flux — their transaction carrier was withdrawn and their fee terms removed upstream — so every claim is classified as specified, designed-but-unspecified, or open, and where ZIP text, fork text, and the implementation disagree (as they do on the split-note nullifier derivation), all three readings are stated with the divergence flagged rather than silently resolved.

Contents

1 Scope, status, and sources

Zcash Shielded Assets (ZSA) is the proposed extension of Orchard that carries user-issued assets in the shielded pool. Two consensus ZIPs define it: ZIP 226, "Transfer and Burn of Zcash Shielded Assets", and ZIP 227, "Issuance of Zcash Shielded Assets" — both Status Draft, Category Consensus, created 2022-05-01, owned by Pablo Kogan and Vivek Arte (QED-it), Daira-Emma Hopwood, and Jack Grigg, with credits to Daniel Benarroch, Aurelien Nicolas, Deirdre Connolly, and Teor, and discussed in the single thread zcash/zips#618 (`zip-0226.rst` and `zip-0227.rst` lines 1–18 @ `69610984`). The combined protocol, OrchardZSA, enables issuance (ZIP 227) and transfer and burn (ZIP 226) of Custom Assets on the Zcash chain; ZIP 227 must only be implemented in conjunction with ZIP 226, and issuance is deliberately transparent — issuance outputs are unencrypted — so that every asset's circulating supply can be tracked on chain (`zip-0226.rst` lines 42–43; `zip-0227.rst` lines 47, 54).

Terminology is ZIP 227's: an *Asset* is a type of note transferable on the Zcash blockchain; ZEC is the default — and currently the only defined — Asset on mainnet, TAZ on testnet; a *Custom Asset* is any Asset other than ZEC and TAZ (`zip-0227.rst` lines 35–39).

The design reduces to one alteration of Orchard's value algebra. The fixed value base V^{Orchard} of the homomorphic Pedersen value commitment (*Ironwood Guide*, §"Conservation without disclosure: value commitments and the binding signature") is replaced, per Asset, by an *Asset Base* witnessed inside the circuit; the same Asset Base must appear in the old and the new note commitments and as the base of the Action's value commitment, so balance holds separately per Asset Identifier without revealing which Assets — or how many distinct ones — a transaction transfers (`zip-0226.rst` lines 59–63). The transparent protocol is unchanged: unshielding is impossible for Custom Assets, burn is the only supply-reduction mechanism and sends to no address, and the Native Asset cannot be burnt (`zip-0226.rst` lines 206–230).

1.1 Scope and deferrals

This volume develops only the delta over Orchard: the per-asset note type and note commitment; the Asset Base and its derivation from an Asset Identifier; the per-asset value commitment, the burn mechanism, and the burn-aware balance equation; split notes, the mechanism that replaces dummy spends for Custom Assets; the changes to the Action circuit; the issuance layer of ZIP 227 — issuance keys, issue notes, reference notes, and the node-side issuance state — and the note-plaintext extension.

Everything the delta rests on is constructed in earlier volumes and is cited, never re-derived: notes and note commitments (*Ironwood Guide*, §"The note and its commitment"); value commitments, the bases V^{Orchard} and R^{Orchard}, and the binding signature (*Ironwood Guide*, §"Conservation without disclosure: value commitments and the binding signature"); the Action statement and dummy spends (*Ironwood Guide*, §"The Action statement, formally"); nullifiers (*Ironwood Guide*, §"The nullifier, and the loop back to minting"); Sinsemilla (*Ironwood Guide*, §"Sinsemilla: hash, commitment, and short forms"); and, at the wallet layer, transaction construction, PCZT, and fees (*Wallet Guide*, §"Transaction construction", §"Partially created transactions (PCZT)", §"Fees (ZIP-317)").

Deployment status is stated once, in §1.2. Subsequent sections construct their objects without requalifying each of them with it.

1.2 Status

Both ZIPs are Status Draft. ZIP 226's Deployment section reads: "The Zcash Shielded Assets protocol is scheduled to be deployed in Network Upgrade 7 (NU7). This ZIP currently assumes that this will be the case" (`zip-0226.rst` lines 425–426). ZIP 227's Deployment section is "TBD", and its Test Vectors and Reference Implementation entries read "LINK TBD" (`zip-0227.rst` lines 661–675).

The network-upgrade reality is thinner than the assumption. ZIP 254, the NU7 deployment ZIP, is Withdrawn and delegates to `draft-arya-deploy-nu7` (Status Draft), which fixes the con-

sensus branch ID `0x77190AD8` and the minimum network protocol versions 170150 (testnet) and 170160 (mainnet), leaves the activation height TBD on both networks, and lists no feature ZIPs at all (`zip-0254.md`, `draft-arya-deploy-nu7.md` @ `69610984`). The repository README's "NU7 Candidate ZIPs" list (218, 230, 231, 233, 234, 235, 2002, 2003, plus a possible ZIP 317 update) does not include ZIP 226 or ZIP 227, still names the withdrawn ZIP 230, and is explicitly non-decisional: "no decision has been made on whether to include each of these ZIPs"; the deployment draft "will define which ZIPs are included in NU7" (`README.rst` lines 70–92 @ `69610984`). In the current zips tree, ZIP-204 assigns NU7 protocol versions 170180 (Testnet) and 170190 (Mainnet), but `draft-arya-deploy-nu7` still says 170150 and 170160; the draft is therefore internally stale even though its activation heights remain TBD (zips `ad30e59`, ZIP-204 and `draft-arya-deploy-nu7`). In the Zcash Foundation's February 2026 NU7 sentiment polls, the proposal that drew "universal or near-universal support" was Tachyon, not ZSA. ZSA's own standing in those polls is split: ZCAP supported ZSAs at 70.4%, the coinholder poll ran 98.6% against, and the Foundation did not recommend ZSAs for NU7, writing that "before any decision is made, we need to understand why coinholders oppose ZSAs so strongly" (zfnd.org, "NU7 Polling Results: What We Heard and Where We Go From Here", 2026-02-23, `https://zfnd.org/nu7-polling-results-what-we-heard-and-where-we-go-from-here/`).

The specification's baseline moved beneath it in 2026. ZIP 226 is now defined relative to the protocol with ZIP 2005 (Ironwood Quantum Recoverability; Status Proposed, deploying with NU6.3 "Ironwood") applied, and states that ZIP 2005 "is expected to deploy before any ZSA activation" (`zip-0226.rst` line 45, which still cites the ZIP under its older name "Orchard Quantum Recoverability"; `zip-2005.md` header). The dependency points upward: if the NU6.3 content changes, the ZSA specification inherits the change.

The carrier format has no live specification. ZIP 226 defines its transaction structure and sighash by deferring to ZIP 230 (the v6 transaction format) and ZIP 246 (the v6 digests) (`zip-0226.rst` lines 373–405, 456–462); both are now Withdrawn. ZIP 230 is retitled "Withdrawn Version 6 Transaction Format" and warns: "This ZIP has been obsoleted by ZIP 229, and will not be deployed. Transaction version number 6 is now defined by ZIP 229" (`zip-0230.rst` lines 5, 14, 37–40; withdrawal commit `04db8968`, 2026-05-16). ZIP 246 is retitled "Digests for the Withdrawn Version 6 Transaction Format" (`zip-0246.rst` lines 4–11, 36–44; commit `00f37219`, merged 2026-07-09). The ZIP 229 that now owns transaction version 6 (Status Draft, created 2026-06-13, for NU6.3) excludes ZSA by declared non-requirement: "The v6 transaction format need not support ZSAs, the zip233Amount field, the explicit fee field, or per-transparent-input sighash information that appeared in the withdrawn ZIP 230, or those features and other extensibility affordances planned for ZIP 248", and "The value 6 for the transaction version number need not avoid collisions with the withdrawn ZIP 230 or with the current draft of ZIP 248" (`zip-0229.md` lines 154–162). ZIP 248, the intended extensible-format home for the ZIP 230/231/246 material, exists only as the unmerged pull request zcash/zips#1156. The rot reaches into the ZSA texts themselves: ZIP 227's SigHash consensus rule cites "modifications described in ZIP 226 [#zip-0226-txiddigest]", an anchor ZIP 226 no longer contains (`zip-0227.rst` lines 474, 693). This volume therefore bins the entire transaction-format layer — wire format, digest tree, sighash — as designed but unspecified (§1.4), even though the cryptographic core above it is specified.

The v6 note-plaintext lead byte is formally unassigned. Occurrences of ZIP 230's constant (originally `0x03`) were changed to the placeholder `{{LEADBYTE}}` "to clarify that this ZIP does not reserve that lead byte value, and to avoid confusion with the different specification of lead byte 0x03 in ZIP 2005"; ZIP 226 correspondingly requires the placeholder `{{ZSALEADBYTE}}` (`zip-0230.rst` lines 41–44; `zip-0226.rst` line 162; commit `8418662`, 2026-05-16). The pinned implementation ships `0x03` (§1.3).

The ZSA fee terms lost their upstream home on 2026-06-26: ZIP 317's change history for that date records "Removed the ZSA issuance and asset-creation fee contributions (ZIP 227)", so ZIP 227's "Changes to ZIP 317" section now points at a fee calculation that contains no ZSA terms (`zip-0317.rst` lines 11–15, 668–676; `zip-0227.rst` lines 604–610). The issuance and asset-creation contributions survive only in the QED-it zips fork and in `librustzcash-zsa` (§1.3); they are binned

designed but unspecified.

Two claims commonly attached to ZSA are stated here at exactly the rigor the sources support. ZIP 226 lists reference implementations — the QED-it forks of zcashd, `orchard`, `librustzcash`, and `halo2` — and test vectors at QED-it/zcash-test-vectors (`zip-0226.rst` lines 428–439). No artifact in the local corpus records an audit of ZSA (a repository-wide search over the ZIP texts and all three QED-it forks finds none), and none documents a testnet deployment; the only local traces of node integration are the `temporary-zebra` cargo feature and the fork ecosystem itself. Both claims therefore rest on web sources, pinned here by URL and date, and both are scoped. Audit: Least Authority TFA GmbH, *OrchardZSA Protocol Security Audit Report* (commissioned by ZCG as Zcash Ecosystem Security Lead), Updated Final Audit Report 30 January 2025. Scope: the QED-it `orchard` `zsa1-vs-main` diff (PR QED-it/orchard#7, including the ZSA circuit extensions) and the QED-it `halo2` gadget changes (PR QED-it/halo2#18), at revisions `a7c02d22` (initial) and `01e85a5f` (verification); findings: no issues, four suggestions (all but one resolved). `https://leastauthority.com/wp-content/uploads/2025/01/Least-Authority-ZCG-OrchardZSA-Protocol-Updated-Final-Audit-Report.pdf`.
The audit does not cover `librustzcash-zsa` or the v6 transaction format layer, and the audited revisions predate the pinned `cf801a5d`. Testnet: ZecHub, *Zcash Shielded News Vol. 61* (2025-12-17): "A feature-complete version of ZSAs is now available on testnet" (`https://zechub.substack.com/p/zcash-shielded-news-vol61`); the testnet is QED-it's public Zebra-based single-node ZSA testnet (see forum.zcashcommunity.com thread #53293, "ZSA Testnet Question"). This volume accordingly asserts "audited" only of the orchard/halo2 circuit layer at the audited revisions, and "running on QED-it's public testnet" of the stack as a whole — and nothing stronger.

1.3 Sources

Table 1 lists the primary sources, each pinned to a commit. All facts in this volume were verified firsthand at the stated commits; the only web sources cited are pinned by URL and date — the Least Authority OrchardZSA audit report (2025-01-30), ZecHub *Shielded News* Vol. 61 (2025-12-17), and the Zcash Foundation's NU7 Polling Results post (2026-02-23), all three in §1.2.

Artifact	Pin / date	Role
`zcash/zips` (upstream)	`69610984`, 2026-07-09	primary ZIP ground truth
`QED-it/orchard`, branch `zsa1`	`cf801a5d`, 2026-06-29	crate `orchard` 0.14.0
`QED-it/librustzcash`, branch `zsa1`	`3f9b53fc`, 2026-04-17	v6 carrier, fee rule
`QED-it/zips`, branch `zsa1`	`fd71419a`, 2026-02-11	fork baseline for diffing

Table 1: Primary ZSA sources, pinned.

Current-head boundary (2026-08-30). The QED-it forks have moved since the source snapshot: orchard `b578ad9` is based on 0.15.0 and adds the NU6.3 cross-address restriction to the ZSA circuit, while librustzcash `c5c232d` synchronises the NU6.2-era upstream transaction changes. Neither change supplies a current normative carrier for ZSA: upstream ZIPs 230 and 246 remain withdrawn, ZIPs 226 and 227 remain Draft, and the NU7 deployment draft still does not select them. Detailed wire, circuit, and fee claims below therefore continue to describe the explicit pins in Table 1; later fork behaviour is not silently attributed to that public-testnet snapshot.

Upstream versus fork. Upstream ZIP 226 diverged from the fork text at `fd71419a` in three substantive ways: it added the ZIP 2005 dependency paragraph; it changed the split-note ψ^{nf} from "sampled uniformly at random on $\mathbb{F}_{p_{\mathsf{Pallas}}}$" to a PRFexpand derivation whose domain-separator byte `0x0A`

is reserved by ZIP 2005; and it replaced the lead byte `0x03` with the `{{ZSALEADBYTE}}` placeholder. ZIP 227 upstream differs from the fork only in punctuation and one reference title.

Code versus ZIP. Where the pinned code disagrees with the pinned ZIP text, this volume presents the ZIP formula and flags the divergence inline. Two content divergences head the list. First, the split-note nullifier randomizer: ZIP 226 derives ψ^{nf} with domain byte `0x0A`, while `orchard-zsa` @ `cf801a5d` computes it with PRFexpand domain `0x09` over a random per-note seed (`src/circuit.rs` lines 316–319) — the older fork-era derivation. Second, the note-plaintext lead byte: a placeholder upstream, but `0x03` in code (`src/primitives/zcash_note_encryption_domain.rs` lines 46–49) — the very value ZIP 2005 now specifies differently. A third divergence is one of status rather than content: the digest personalisations in `orchard-zsa` match the withdrawn ZIP 246 exactly, and `librustzcash-zsa` implements `TxVersion::V6` with the withdrawn ZIP 230 wire format — the entire carrier layer the code implements is specified only by withdrawn ZIPs (§1.2). Further code-versus-text divergences confirmed at the same pins — the omitted explicit-fee digest field, the missing memo branch and its non-spec substitute digest, the empty-action issuance rule, and the valueBurn bound wording — are inventoried in §6.6, §3.7, and §4.4.

Pin consistency. The two implementation forks are not mutually consistent snapshots: `librustzcash-zsa` @ `3f9b53fc` patches `orchard` to QED-it rev `77d3274c`, older than the `orchard-zsa` checkout head `cf801a5d`; it also pins `sapling-crypto` @ `59535fb5` and `zcash_spec` @ `d5e84264`, while `orchard-zsa` pins `halo2_proofs/halo2_gadgets` @ `ef3d0ba2` and the same `zcash_spec` (`Cargo.toml` lines 234–239 and 118–123 respectively). Each fork is cited at its own pin; where the gap between them is load-bearing, the text says so.

Gating. In `librustzcash-zsa`, all ZSA and v6 code sits behind `cfg(zcash_unstable = "nu7")`; in `orchard-zsa`, issuance sits behind the cargo feature `zsa-issuance` (which pulls in `secp256k1`) and node integration behind `temporary-zebra`.

Non-sources. The branch `ltf/zsa` of upstream `zcash/orchard` points at `a799082e` (2025-11-25), an ancestor of `main`, and contains no ZSA code; it is not usable ground truth. Node-side enforcement is a second gap: ZIP 227 specifies chained per-node issuance state (the `issued_assets` map), but no node implementation is checked out locally, so the supply and burn logic verified for this volume is library-side only (`orchard-zsa` `src/bundle/burn_validation.rs`, `src/issuance.rs`).

1.4 Method: the three bins for ZSA

This volume adopts the series' classification of design-stage claims (*Voting Guide*, §"The source situation: code as the de facto specification"): every claim is *specified*, *designed but unspecified*, or an *open problem*, and the bin stays visible in the prose. The distribution differs sharply from the Tachyon volume's. Bin (i) holds the entire cryptographic core — the per-asset note and its commitment, the Asset Base derivation, value commitments and the balance equation, split notes, the circuit statement, issuance keys, issue notes and reference notes, the issuance state machine, and burn — because the ZIP texts state it at consensus rigor and the pinned implementation cross-checks it. Bin (ii) holds the carrier layer: the v6 wire format, the digest tree and sighash, the note-plaintext lead byte, and the ZSA fee terms, each specified today only by withdrawn ZIPs, the QED-it fork texts, or code (§1.2). Bin (iii) holds NU7 inclusion and activation, the eventual lead-byte assignment, and the carrier format's future home. Per the series conventions, every formula in this volume is transcribed verbatim from the pinned ZIP text or the pinned code, and the citation names which.

2 Asset identity

An OrchardZSA note denominates value in one of many assets sharing a single shielded pool, so the protocol must attach to every note an asset identity satisfying two demands. First, uniqueness: "The Asset identification should be unique (among all shielded pools) and different issuer public keys should not be able to generate the same Asset Identifier" (`zip-0227.rst`:76). Second, usability inside the circuit: the identity must ultimately be a Pallas point, because it serves as the asset-specific base of the value commitment (§2.4). ZIP 227 meets both demands with one derivation chain,

$$\texttt{asset\_desc} \longrightarrow \mathsf{assetDescHash};$$
$$(\mathsf{issuer}, \mathsf{assetDescHash}) \longrightarrow \mathsf{AssetId} \longrightarrow \mathsf{AssetDigest} \longrightarrow \mathsf{AssetBase},$$

which the ZIP itself presents as a relation diagram (`zip-0227.rst`:210–224): the issuer-chosen description string is hashed, the hash paired with the issuer key into an identifier, the identifier hashed into a digest, and the digest mapped onto the Pallas curve. Every Asset issued under ZIP 227 is scoped to the issuer that issued it, and its Asset Identifier is globally unique, "used across all Zcash protocols that support ZSAs" (`zip-0227.rst`:227–240).

This section constructs the chain end to end: the issuance key pair at its root (§2.1); the hash steps from description to identifier (§2.2); the group hash from identifier to base point (§2.3); the native asset's special position outside the chain (§2.4); and the parts of the chain consensus sees on the wire versus recomputes (§2.5). Deployment status for the whole protocol is fixed once, in §1.2, and not restated per object.

2.1 The issuance key pair

The OrchardZSA protocol adds exactly two keys to the key material of Orchard: the *issuance authorizing key* isk, which authorizes the issuance of Asset Identifiers by a given issuer and is used only by that issuer, and the *issuance validating key* ik, which validates issuance transactions (`zip-0227.rst`:84–91). Neither key touches the spend path. The ZIP's rationale: the issuance key structure is independent of the original key tree but derived analogously via ZIP 32, which keeps issuance details and Asset Identifiers consistent across multiple shielded pools, and separates issuance authority from spend authority — allowing a potential transfer of issuance authority without compromising spend authority (`zip-0227.rst`:524).

Definition 2.1 (Issuance authorization signature scheme)**.** The scheme IssueAuthSig is instantiated as a BIP-340 Schnorr signature over the secp256k1 curve, with constants set per the secp256k1 standard parameters as described in BIP 340. Its associated types are: messages are arbitrary byte sequences; signatures are 64-byte sequences or $\perp$; public keys are 32-byte sequences or $\perp$; private keys are 32-byte sequences (`zip-0227.rst`:122–135). Signing fixes the BIP-340 auxiliary data to $a = [\texttt{0x00}]^{32}$: the signature is $\sigma := \mathsf{Sign}(\mathsf{isk}, M)$ with auxiliary data a, or $\perp$ if Sign fails; on the wire it is encoded as $\texttt{issueAuthSig} = [\texttt{0x00}] \,||\, \sigma$ (`zip-0227.rst`:190–207).

In the implementation, type `ZSASchnorr` realizes the scheme with `ALGORITHM_BYTE = 0x00`, secret keys as `secp256k1::SecretKey`, public keys as `XOnlyPublicKey`, and signing via `sign_schnorr_with_aux_rand` with the all-zero 32-byte auxiliary array (`orchard-zsa src/issuance/auth.rs`:163–273 @ `cf801a5d`).

Construction 2.2 (Issuance key derivation)**.** The issuance authorizing key is generated by ZIP 32's hardened-only key derivation — the maps MKGh and CKDh, constructed in the *Wallet Guide*, §"Hardened-only derivation", and not re-derived here — instantiated in a dedicated Issuance context (`zip-0227.rst`:139–166). Let S be a seed of at least 32 and at most 252 bytes.

(1) *Context.* The issuance context sets Issuance.MKGDomain = `ZcashSA_Issue_V1` and Issuance.CKDDomain = `0x81` (`zip-0227.rst`:148–149).

(2) *Master key.* Set $m_{\mathsf{Issuance}} := \mathsf{MKGh}^{\mathsf{Issuance}}(S)$.

(3) *Child derivation.* Set $\mathsf{CKDsk}((\mathsf{sk}_{\mathrm{par}}, c_{\mathrm{par}}), i) := \mathsf{CKDh}^{\mathsf{Issuance}}((\mathsf{sk}_{\mathrm{par}}, c_{\mathrm{par}}), i)$. The two-argument call fixes the general form's *lead* = 0 and *tag* = [ ], exactly the case whose suffix the ZIP-32 encoding omits (`zip-0032.rst`:449–462; the *Wallet Guide*'s remark "The general form: triples, and sibling instantiations").

(4) *Path.* Derive along m_{Issuance} / purpose′ / coin_type′ / account′, with purpose fixed to 227 (`0xe3`; hardened `0x800000e3` per BIP 43), coin_type as in the ZIP 32 key path levels, and account fixed to index 0 (`zip-0227.rst`:160–164).

(5) *Key extraction.* From the generated pair (sk, c), set isk := sk (`zip-0227.rst`:164–166).

The implementation realizes the context as a struct `Issuance` implementing the `zip32` crate's hardened-only `Context` trait, with `MKG_DOMAIN` set to `ZcashSA_Issue_V1` and with `CKD_DOMAIN` reusing the Orchard byte, constant `PrfExpand::ORCHARD_ZIP32_CHILD`; the seed-level constructor `from_zip32_seed` derives along $[227', \text{coin\_type}', 0']$ and rejects any non-zero account with error `NonZeroAccount` (`orchard-zsa src/issuance/auth.rs`:38–99, 219–236 @ `cf801a5d`). The `zip32` crate 0.2.0 implements MKGh and CKDh exactly, and its `derive_child(index)` calls `ckdh_internal(index, 0, [])` — *lead* = 0, *tag* = [ ], omitted from the PRF message (`src/hardened_only.rs`:95–130).

Remark 2.3 (Domain separation between the Orchard and Issuance trees)**.** The byte Issuance.CKDDomain = `0x81` equals Orchard.CKDDomain (`zip-0032.rst`:476), and the implementation reuses `PrfExpand::ORCHARD_ZIP32_CHILD` (`zcash_spec src/prf_expand.rs`:119 @ `d5e84264`). Separation between the two ZIP-32 trees therefore rests entirely on the differing MKGDomain: distinct master secret keys and chain codes feed every subsequent PRFexpand call. Neither ZIP states whether the byte reuse is deliberate; ZIP 32 asks only that CKDDomain values be tracked as part of the global set of domains defined for PRFexpand. The derivation is *specified*; we flag the shared byte because the separation argument is easy to misattribute to it.

Definition 2.4 (Issuance validating key and issuer identifier)**.** The issuance validating key is ik := PubKey(isk), where PubKey is the BIP-340 algorithm and the output is in big-endian order as defined in BIP 340; the derivation returns ⊥ if PubKey fails, which happens with very low probability, in which case the keys are discarded and derivation repeated with a different isk (`zip-0227.rst`:168–183). Its encoding prepends a signature-scheme byte, which MUST be `0x00`, indicating BIP 340:

$$\mathsf{ik\_encoding} = [\texttt{0x00}] \,\|\, \mathsf{ik} \qquad (33 \text{ bytes}),$$

and the issuer identifier is defined by issuer = ik_encoding (`zip-0227.rst`:103–110, 178–179). The encoding currently appears on chain only in the `issuer` field of an issuance bundle (`zip-0227.rst`:178–180).

Remark 2.5 (Permanence of the issuer binding)**.** ZIP 227 notes that "the equality of `issuer` and `ik_encoding` might not hold in future when key rotation is specified for issuance key pairs" (`zip-0227.rst`:103–110). No rotation mechanism exists today: an asset's identity is permanently bound to the one ik under which it was issued, and the ZIP's remedy for key compromise is to finalize the asset and reissue under a new Asset Identifier: "If an issuer identifier is compromised, the finalize boolean for all the Assets issued with that key should be set to 1; then the issuer should change to a new issuance authorizing key (hence a new issuer identifier), and issue new Assets, each

with a new AssetId" (zip-0227.rst:640–645, Security and Privacy Considerations, "Issuance Key Compromise"). This is a *stated limitation* of the specified design; the rotation mechanism itself is an open design area.

2.2 From description to identifier

An issuer names each of its assets with an *asset description* `asset_desc`: a non-empty byte sequence, which SHOULD be a well-formed UTF-8 code unit sequence according to Unicode 15.0.0 or later (zip-0227.rst:237–241). Within the scope of an issuer, Asset Identifier uniqueness is obtained by way of the asset description (zip-0227.rst:237–240). The current ZIP imposes no maximum length — its rationale notes that a description "can be longer than 512 bytes without incurring chain costs", the 512-byte cap being an earlier design (zip-0227.rst:528–539). The description is also global: ZIP 226 states that "Every Asset is defined by an Asset description, `asset_desc`, which is a global byte string (scoped across all future versions of Zcash)", so that in a future upgrade the same string can carry over to a different shielded pool, with nodes transforming the identifier chain between pools without balance violations (zip-0226.rst:93–103).

Definition 2.6 (Asset Identifier)**.** The description is hashed,

$$\mathsf{assetDescHash} := \text{BLAKE2b-256}(\texttt{ZSA-AssetDescCRH}, \texttt{asset\_desc})$$

(zip-0227.rst:243–247), and the Asset Identifier is the pair

$$\mathsf{AssetId} := (\mathsf{issuer}, \mathsf{assetDescHash})$$

(zip-0227.rst:249–253), with the canonical encoding

$$\mathsf{EncodeAssetId}(\mathsf{AssetId}) := [\texttt{0x00}] \,\|\, \mathsf{issuer} \,\|\, \mathsf{assetDescHash} \qquad (66 = 1 + 33 + 32 \text{ bytes}),$$

where the initial `0x00` is a version byte enabling future issuance protocols (zip-0227.rst:255–261). The version byte is not to be confused with the byte that follows it — the first byte of issuer, which indicates the signature scheme (Definition 2.4); every current encoding happens to begin with two `0x00` bytes of distinct meaning.

The pair structure carries the uniqueness argument. Because the issuer identifier is a component of AssetId, two different issuers cannot produce the same identifier; in the ZIP's words, the Custom Asset "is described via a combination of the issuer identifier and an asset description string, to preclude the possibility of two different issuers creating colliding Custom Assets" (zip-0227.rst:525). Within one issuer, distinct descriptions separate assets via assetDescHash; for every new Asset there MUST be a new and unique Asset Identifier (zip-0226.rst:93–96).

Remark 2.7 (Why a hash, not the string)**.** ZIP 227 replaces the direct description string of an earlier design with a collision-resistant hash for five stated reasons (zip-0227.rst:528–559): a fixed 32 bytes per Issue Action costs less bandwidth than up to 512 bytes; descriptions can exceed 512 bytes without chain cost; carrying the byte string on chain would not make the user-visible description consensus-visible anyway (the string could itself be a hash of an off-chain description); absent key rotation, marking an issuer as trusted is insufficient — each Asset Identifier needs independent out-of-band verification that conveys the description; and an on-chain description is an "attractive nuisance" — hashing forces wallets to obtain the description from a trusted party, a registry, or direct user approval. Consistent with the last point, wallets MUST NOT display just the `asset_desc` string as the name of the Asset; the ZIP suggests a petname system mapping names to Asset Identifiers via client configuration, or the Asset Digest as a compact unique byte sequence, e.g. in QR codes (zip-0227.rst:263–266).

Implementation. The identifier is the enum variant `AssetId::V0`, holding the validating key and the 32-byte description hash; `encode_asset_id` emits the version byte `0x00`, the 33-byte key encoding, and the hash — 66 bytes, matching EncodeAssetId (`orchard-zsa src/note/asset_base.rs`:20–60 @ `cf801a5d`). The function `compute_asset_desc_hash` enforces non-emptiness through its argument type `NonEmpty<u8>` and *panics* if the description is not well-formed UTF-8 (`orchard-zsa src/issuance.rs`:135–154 @ `cf801a5d`). The specification says SHOULD; the implementation enforces a hard MUST at its API boundary. Since only the 32-byte hash is consensus-visible, neither variant is consensus-enforceable: we classify the UTF-8 constraint as an issuer-side convention — *specified* as a SHOULD, with the stricter behaviour an implementation choice.

2.3 From identifier to base point

Two further steps land the identifier on the Pallas curve. The full chain, collected:

$$\begin{aligned}
\mathsf{assetDescHash} &:= \text{BLAKE2b-256}(\texttt{ZSA-AssetDescCRH},\ \texttt{asset_desc}),\\
\mathsf{AssetId} &:= (\mathsf{issuer}, \mathsf{assetDescHash}),\\
\mathsf{AssetDigest}_{\mathsf{AssetId}} &:= \text{BLAKE2b-512}(\texttt{ZSA-Asset-Digest},\ \mathsf{EncodeAssetId}(\mathsf{AssetId})),\\
\mathsf{AssetBase}_{\mathsf{AssetId}} &:= \mathsf{ZSAValueBase}(\mathsf{AssetDigest}_{\mathsf{AssetId}}),
\end{aligned}$$

where the digest is defined at `zip-0227.rst`:268–278, the base at `zip-0227.rst`:280–290, and the OrchardZSA instantiation of the final step is

$$\mathsf{ZSAValueBase}(\mathsf{AssetDigest}) := \mathsf{GroupHash}^{\mathsf{Pallas}}(\texttt{z.cash : OrchardZSA},\ \mathsf{AssetDigest})$$

(`zip-0227.rst`:292–301). The subscript AssetId may be omitted when clear from context (`zip-0227.rst`:224). The group hash is the protocol specification's $\mathsf{GroupHash}^{\mathbb{P}}$ (§5.4.9.8, "Group Hash into Pallas and Vesta"), whose construction — `expand_message_xmd` over BLAKE2b, two field elements through the simplified SWU map, summed and carried through the isogeny — the *Ironwood Guide* develops in "GroupHash, domain separation, and nothing-up-my-sleeve generators"; we reuse it as a deterministic, public, nothing-up-my-sleeve map into the Pallas group and do not re-derive it here.

ZIP 226 types the result as a *non-trivial* point: AssetBase is "the unique element of the Pallas group that identifies each Asset in the Orchard protocol ... a valid group element that is not the identity and is not $\bot$" (`zip-0226.rst`:111–117), a non-identity point of $\mathcal{E}(\mathbb{F}_{p_{\mathsf{Pallas}}})$ in our notation. Its byte form is

$$\texttt{asset\_base} := \mathrm{LEBS2OSP}_{\ell_{\mathbb{P}}}(\mathrm{repr}_{\mathbb{P}}(\mathsf{AssetBase})),$$

the 32-byte representation stored in note plaintexts and burn fields (`zip-0226.rst`:114; `zip-0230.rst`:540–545). ZIP 226 notes the encoding assumes canonicity, "true for the Pallas group, but may not hold for future shielded protocols" (`zip-0226.rst`:111–117). Table 2 collects the sizes along the chain.

Object	Bytes	Form	Source
ik	32	x-only BIP-340 key, big-endian	`zip-0227.rst`:130–133
ik_encoding = issuer	33	[0x00] ‖ ik	`zip-0227.rst`:178–179
assetDescHash	32	BLAKE2b-256 output	`zip-0227.rst`:243–247
EncodeAssetId(AssetId)	66	[0x00] ‖ issuer ‖ assetDescHash	`zip-0227.rst`:255–259
AssetDigest	64	BLAKE2b-512 output	`zip-0227.rst`:268–278
`asset_base`	32	$\mathrm{LEBS2OSP}_{\ell_{\mathbb{P}}}(\mathrm{repr}_{\mathbb{P}}(\cdot))$	`zip-0230.rst`:540–545

Table 2: Byte sizes along the asset-identity derivation chain.

Why the base must be a genuine group-hash output. The chain's final step is not a formality. In a transfer, the output note of a split Action cannot be allowed to carry an arbitrary Pallas point as its base: "We must enforce it to be an actual output of a GroupHash computation ... Without this enforcement the prover could input a multiple (or linear combination) of an existing Asset Base, and thereby attack the network by overflowing the ZEC value balance and hence counterfeiting ZEC funds" (`zip-0226.rst`:299–307). The circuit therefore ensures the value base points of all output notes are actual GroupHash outputs "as they originate in the Issuance protocol which is publicly verified", and ZIP 226 adds: "We prevent a potential malleability attack on the Asset Identifier by ensuring the output notes receive an Asset Base that exists on the global state" (`zip-0226.rst`:104–106, 299–307). The enforcement mechanism belongs to the transfer sections of this volume (§5).

Remark 2.8 (The identity-point corner case)**.** The protocol specification's $\mathsf{GroupHash}^{\mathbb{G}}(D, M)$ has the full group as codomain, so its output can in principle be the zero point (`protocol.tex` §5.4.9.8; the *Crypto Guide* constructs the hash-to-curve pipeline). ZIP 227's ZSAValueBase never states what happens in that event; the non-identity requirement comes from ZIP 226's typing of the note field (`zip-0226.rst`:111–117) and from the implementation, which panics inside `AssetBase::custom` ("this will happen with negligible probability"), rejects the identity in `AssetBase::from_bytes`, and carries an `Error::AssetBaseCannotBeIdentityPoint` check in `IssueAction::verify` that the panic renders unreachable (`orchard-zsa src/note/asset_base.rs`:98–138, `src/issuance.rs`:216–244 @ `cf801a5d`). Whether the intended consensus behaviour is "reject the transaction" or "the issuer must pick a different description" is unspecified in the ZIP text. The event has negligible probability; the specification gap is nevertheless real, and we classify the corner case as *open*.

2.4 The native asset

ZEC is the default — and currently the only defined — Asset for mainnet, TAZ for testnet; a *Custom Asset* is any Asset other than ZEC and TAZ (`zip-0227.rst`:35–39). The native asset does not pass through the derivation chain at all. Its base is fixed by definition: "the Asset Base of the ZEC Asset will be kept as the original value base point, V^{Orchard}" (`zip-0226.rst`:98); for ZEC,

$$\mathsf{AssetBase}_{\mathsf{AssetId}} := V^{\mathsf{Orchard}},$$

"so that the value commitment for ZEC notes is computed identically to the Orchard protocol deployed in NU5. As such $\mathsf{ValueCommit}^{\mathsf{Orchard}}_{\mathsf{rcv}}(v)$ as defined in ZIP 224 is used as $\mathsf{ValueCommit}^{\mathsf{OrchardZSA}}_{\mathsf{rcv}}(V^{\mathsf{Orchard}}, v)$" (`zip-0226.rst`:182–194). The point $V^{\mathsf{Orchard}} := \mathsf{GroupHash}^{\mathsf{Pallas}}$(`z.cash : Orchard-cv`, `"v"`) is already constructed, together with its companion R^{Orchard}, in the *Ironwood Guide*, "Conservation without disclosure: value commitments and the binding signature", Definition "Net value commitment"; we cite it and do not re-derive it.

Two consequences follow. First, indistinguishability: ZIP 230 states that "An Orchard note is essentially equivalent to an OrchardZSA note with $\mathsf{AssetBase} = V^{\mathsf{Orchard}}$; in particular, they have the same note commitment ... and need not be distinguished in note plaintexts" (`zip-0230.rst`:531–538), and ZIP 226's privacy discussion states that OrchardZSA note commitments for the native asset are indistinguishable from those for non-native Assets (`zip-0226.rst`:83–84). Second, exclusion from burning: the first consensus rule for `assetBurn` requires $\mathsf{AssetBase} \neq V^{\mathsf{Orchard}}$ for every $(\mathsf{AssetBase}, v) \in$ `assetBurn` (`zip-0226.rst`:220–223) — the native asset cannot be burnt.

Implementation. The native base is `AssetBase::zatoshi()`, computed as the Pallas hash-to-curve over personalization `"z.cash:Orchard-cv"` of the message `"v"` — exactly V^{Orchard}; `AssetBase::is_zatoshi()` is a constant-time equality with that point; and `ValueCommitment::derive` computes $[v]V + [\mathsf{rcv}]R$ with $V =$ `asset.cv_base()` and R the fixed base R^{Orchard}, so a ZEC note's value commitment is bit-identical to NU5 Orchard's (`orchard-zsa` @ `cf801a5d`: `note/asset_base.rs`:140–155, `constants/fixed_bases.rs`:35–45, `value.rs`:380–400).

2.5 Identity on the wire

Of the whole chain, consensus sees exactly two links on the issuance side: the issuer and the description hash. The v6 transaction's issuance bundle carries `issuerLength` (compactSize), `issuer` (the issuer identifier per ZIP 227), `nIssueActions` (compactSize), `vIssueActions`, and `issueAuthSig` — the signature bytes preceded by a `0x00` byte indicating BIP 340. The first four fields are always present; `issueAuthSig` is present iff `nIssueActions` > 0, and if `nIssueActions` $= 0$ then `issuerLength` MUST be 0 (`zip-0230.rst`:166–179, 215–217, 430–443). Each Issue Action carries `assetDescHash` as `byte[32]`, `nNotes` (compactSize, possibly zero, to finalize without issuing), `vNotes` (115 bytes per note description: 43-byte recipient, 8-byte value, 32-byte ρ, 32-byte rseed), and `flagsIssuance` with finalize in the least significant bit; the burn encoding carries `assetBase` as `byte[32]` (`zip-0230.rst`:367–383, 446–494). On the transfer side, the v6 Orchard note plaintext inserts the asset between rseed and the memo key (the 32-byte key that replaces the in-band 512-byte memo under ZIP-231 and from which the recipient derives the key decrypting its memo in the transaction-level memo bundle; §4.7),

$$[\texttt{leadByte}] \,||\, \mathrm{LEBS2OSP}_{88}(d) \,||\, \mathrm{I2LEOSP}_{64}(v) \,||\, \mathsf{rseed} \,||\, \texttt{asset\_base} \,||\, \mathsf{K}^{\mathsf{memo}}$$

(`zip-0230.rst`:523–553).

The consensus rules bind identity to validation: the ik_encoding used to validate `issueAuthSig` MUST be taken from the `issuer` field and MUST start with byte `0x00`, and for each Issue Action, the Issue Note's AssetBase is derived from the `assetDescHash` field of the action and the `issuer` field of the bundle (`zip-0227.rst`:483–491). The implementation makes the recomputation structural. Function `read_bundle` decodes the issuer via `IssueValidatingKey::decode`, checking the algorithm byte; function `read_action` reads the 32-byte hash and recomputes the base as `AssetBase::custom` of `AssetId::new_v0(ik, asset_desc_hash)`. The Asset Base is never carried on the wire for issuance, always re-derived (`librustzcash-zsa` @ `3f9b53fc`, crate `zcash_primitives`, file `transaction/components/issuance.rs`, lines 22–90). Redundantly with construction, `IssueAction::verify` checks that every note's base equals the one derived from the validating key and the description hash, failing with error `IssueBundleIkMismatchAssetBase` otherwise (`orchard-zsa` `src/issuance.rs`:216–244 @ `cf801a5d`). An asset's identity is therefore not an on-chain object at all: it is a recomputable function of two consensus-visible byte strings, anchored to the issuer key that signs the bundle.

Classification. The entire derivation chain of this section is *specified*: every formula above appears in the ZIP texts at rigor and is realized by the pinned implementation forks at the pinned commits (`orchard-zsa` @ `cf801a5d`, `librustzcash-zsa` @ `3f9b53fc`). Both ZIPs are Status Draft, Category Consensus, owned by Pablo Kogan, Vivek Arte, Daira-Emma Hopwood, and Jack Grigg, created 2022-05-01 and discussed in zcash/zips issue #618 and PR #680 (`zip-0226.rst`:1–18, `zip-0227.rst`:1–18); deployment status is stated in the scope section. Three reference gaps qualify the bin. First, ZIP 227's Test Vectors and Reference Implementation sections read "LINK TBD" and its Deployment section "TBD" (`zip-0227.rst`:661–675); the de-facto assertion-checked vectors live in the fork: file `test_vectors/asset_base.rs`, generated from the zcash-hackworks test-vectors script `orchard_zsa_asset_base.py`, is replayed by a test that recomputes the full chain from (ik_encoding, `asset_desc`) to `asset_base` (`src/note/asset_base.rs`:236–261 @ `cf801a5d`), with companion vectors for the signature scheme and the ZIP-32 derivation in `issuance_auth_sig.rs` and `zip32.rs`. The vectors' fixed `[u8; 512]` description arrays are a relic of the dropped 512-byte cap and imply no bound. Second, the corner cases flagged above remain: the identity-point behaviour is *open* (Remark 2.8), the UTF-8 constraint is a SHOULD with a stricter implementation (§2.2), and key rotation is an open design area (Remark 2.5). Third, upstream `zcash/zips` @ `69610984` is the text of record; the QEDit zips fork (`zips-zsa` @ `fd71419a`) is essentially identical for the asset-identity material, and the differences — upstream's later additions of a ZIP-2005 dependency, a nullifier ψ^{nf} derivation, and a lead-byte rule — do not touch this chain.

3 Issuance and finalization (ZIP-227)

The preceding section moved Custom Assets between shielded notes; this one brings them into existence. ZIP-227 (Status Draft, Category Consensus; owners Pablo Kogan and Vivek Arte of QEDit, Daira-Emma Hopwood, and Jack Grigg) specifies the issuance mechanism for Custom Assets on the Zcash chain: a per-transaction *issuance bundle* in which a publicly named issuer creates new notes, a per-Asset *finalize* switch that ends issuance forever, and a node-maintained global supply map. The draft's Test Vectors, Reference Implementation, and Deployment sections all read TBD (ZIP-227, header and closing sections @ zips `6961098`). Deployment standing — Draft ZIPs, externally audited at circuit scope, running on QED-it's public testnet, contested for NU7 — is fixed once in §1 and not relitigated here.

Issuance is deliberately *not* shielded. The ZIP's Motivation states: "This ZIP only enables *transparent* issuance. As a first step, transparency will allow for proper testing of the applications that will be most used in the Zcash ecosystem, and will enable the supply of Assets to be tracked" (ZIP-227, *Motivation*). The issuer, the Asset, the issued amounts, and the finalization status are all consensus-visible; only the recipients of the issued notes are shielded, because the notes themselves enter the Orchard note commitment tree (§3.3).

Encoding ground truth. One sourcing decision governs every wire format and digest below, and we state it once. ZIP-227 normatively cites ZIP-230 for all issuance encodings and ZIP-246 for all issuance digests, yet upstream both are Withdrawn: ZIP-230 is the "Withdrawn Version 6 Transaction Format", obsoleted by ZIP-229, its lead byte replaced by the placeholder `{{LEADBYTE}}`, and ZIP-246 carries "Digests for the Withdrawn Version 6 Transaction Format", with v6 digests now assigned to ZIP-229 (ZIP-230 and ZIP-246, headers and warnings @ zips `6961098`) — and ZIP-229 carries no issuance-bundle fields. The QEDit fork of the ZIPs repository retains ZIP-230 and ZIP-246 at Status Draft with lead byte `0x03`, and its copy of ZIP-227 differs from upstream only cosmetically (zips-zsa @ `fd71419`). We therefore present the ZIP-230/246 issuance encoding as what it is: the fork-specified design, deployed on QED-it's public testnet (ZecHub Vol. 61, 2025-12-17; §1.2) and implemented by the pinned fork libraries (the circuit layer audited by Least Authority, 2025-01-30), with no merged upstream v6 specification behind it. The NU7 standing of the whole format stack is part of the deployment picture of §1.

Implementation pins. Claims about deployed behavior cite the QEDit libraries at fixed commits: `orchard-zsa` branch `zsa1` @ `cf801a5d` (2026-06-29), `librustzcash-zsa` branch `zsa1` @ `3f9b53fc` (2026-04-17), and the patched `zcash_spec` (QED-it fork @ `d5e8426`) that carries the new PRF domain byte; upstream `orchard`'s branch `ltf/zsa` is context only, not ground truth (§1.3, *Non-sources*). All `librustzcash-zsa` transaction-format code sits behind `cfg(zcash_unstable = "nu7")`; file paths we cite within that repository are relative to `zcash_primitives/src/`. Except where flagged inline, everything in this section is *specified*: ZIP text at rigor, mirrored by these libraries. Three flags recur: fork-only specification (the wire format above, and fees, §3.9); library-versus-ZIP divergence, noted where it occurs; and node-side enforcement of global state, which no local repository exhibits (§3.7).

3.1 Issuance keys and the issuer identifier

Issuance authority is a new capability, deliberately disjoint from spend authority. The rationale in the ZIP: the issuance key tree is independent of the spending key tree but derived analogously (via ZIP 32), so an organization can delegate issuance without exposing funds, and a future scheme can replace the issuance leaf with a multisignature without touching spending keys (ZIP-227, *Rationale*, and *Issuance Key Derivation*).

The key material itself is constructed once, in §2.1: the BIP-340 signature scheme (Definition 2.1), the hardened-only ZIP-32 derivation of isk in the $\mathsf{Issuance}$ context (Construction 2.2), and the validating key with the issuer identifier, $\mathsf{ik} := \mathit{PubKey}(\mathsf{isk})$ and $\mathsf{issuer} = \mathsf{ik\_encoding} = [\mathtt{0x00}] \,\|\, \mathsf{ik}$

(Definition 2.4). We restate only the parameters the issuance layer consumes: seed S of 32 to 252 bytes, $\mathsf{Issuance.MKGDomain}$ = `ZcashSA_Issue_V1`, $\mathsf{Issuance.CKDDomain}$ = `0x81`, hardened path m_{Issuance} / 227′ / *coin_type*′ / 0′, and $\mathsf{isk} := \mathsf{sk}$ at the leaf (ZIP-227, *Issuance Key Derivation*). No rotation exists today: the on-chain issuer name is the validating key itself, and the ZIP notes the equality might not hold in future if key rotation is ever specified (Remark 2.5; §3.8).

3.2 Asset identity: description, digest, and Asset Base

Every Custom Asset is named by its issuer plus a human-meaningful description, and consumed by the protocol as a Pallas point. The full derivation chain — assetDescHash, AssetId, EncodeAssetId with its version byte, AssetDigest, and AssetBase = ZSAValueBase(AssetDigest) — is constructed once, in §2.2 and §2.3 (Definition 2.6); the hash-not-string rationale is Remark 2.7, and the UTF-8 SHOULD-versus-panic delta between ZIP and implementation is recorded in §2.2. The native Asset bypasses the chain entirely: its base is the Orchard value-commitment base V^{Orchard} (§2.4). An Asset Base equal to the Pallas identity is invalid everywhere — `AssetBase::custom` panics after hashing to the curve and `AssetBase::from_bytes` rejects the identity on parse (Remark 2.8). What the issuance layer adds to the chain is consensus-side recomputation: validators re-derive every Issue Note's AssetBase from the bundle's `issuer` and the action's `assetDescHash`, never reading it off the wire (§2.5; consensus rules, §3.7).

3.3 Issue notes and the derived ρ

Definition 3.1 (Issue note)**.** An *Issue Note* is a tuple $(\mathsf{d}, \mathsf{pk_d}, \mathsf{v}, \mathsf{AssetBase}, \rho, \mathsf{rseed})$,

$$\mathsf{Note}^{\mathsf{Issue}} := \{0,1\}^{\ell_{\mathsf{d}}} \times \mathsf{KA}^{\mathsf{Orchard}}.\mathsf{Public} \times \{0 \,..\, 2^{\ell_{\mathsf{value}}} - 1\} \times \mathcal{E}(\mathbb{F}_{p_{\mathsf{Pallas}}})^* \times \mathbb{F}_{p_{\mathsf{Pallas}}} \times \{0,1\}^{8\cdot 32},$$

a diversifier and diversified transmission key as in Orchard, a value v with $\ell_{\mathsf{value}} = 64$, a non-identity Asset Base, the nullifier-input field element ρ, and a 32-byte rseed that MUST be sampled uniformly at random by the issuer (ZIP-227, *Issue Note*; the ZIP writes the fourth and fifth components $\mathbb{P}^*$ and $\mathbb{F}_{q_{\mathbb{P}}}$, our $\mathcal{E}(\mathbb{F}_{p_{\mathsf{Pallas}}})^*$ and $\mathbb{F}_{p_{\mathsf{Pallas}}}$).

Issue notes never appear on chain as commitments: the wire format carries the note fields themselves (§3.4), and each validator computes the note commitment — by the OrchardZSA NoteCommit of §4 — and appends it to the Orchard note commitment tree. Precisely because the nullifier key of the recipient is unknown to validators, the note's future spend cannot be linked to the issuance transaction (ZIP-227, *Issue Note* and *Issuance Protocol*). Issuance is thus transparent at creation and shielded from the first spend onward.

Computation of ρ. An Orchard note takes $\rho := \mathsf{nf}_{\mathsf{old}}$ from the Action that creates it (the *Ironwood Guide*, §"Delivery: the note reaches its owner"); an issue note has no such Action, so ZIP-227 derives ρ from the transaction's OrchardZSA transfer bundle instead:

$$\mathsf{DeriveIssuedRho}(\mathsf{nf}, i_{\mathsf{A}}, i_{\mathsf{N}}) := \mathsf{ToBase}\big(\mathsf{PRFexpand}_{\mathsf{nf}}\big(\texttt{0x84} \,\|\, \mathrm{I2LEOSP}_{32}(i_{\mathsf{A}}) \,\|\, \mathrm{I2LEOSP}_{32}(i_{\mathsf{N}})\big)\big),$$

$$\rho := \mathsf{DeriveIssuedRho}\big(\mathsf{nf}_{0,0},\ \mathsf{index}_{\mathsf{Action}},\ \mathsf{index}_{\mathsf{Note}}\big),$$

with the PRF keyed by nf as 32 little-endian bytes ($\mathrm{I2LEOSP}_{256}$), where $\mathsf{nf}_{0,0}$ is the nullifier of the input note in the *first* Action of the *first* Action Group of the OrchardZSA bundle (the v6 format partitions a bundle's Actions into one or more Action Groups, each carrying its own anchor and proof; §6.2), and the two indices are the zero-based positions of the Issue Action within the issuance bundle and of the note within its action (ZIP-227, *Computation of ρ*). The domain-separator byte `0x84` extends the PRFexpand lead-byte registry (the *Wallet Guide*, §"The PRFexpand lead-byte registry"); the patched `zcash_spec` carries it as `ORCHARD_DERIVED_ISSUE_RHO` (QED-it `zcash_spec`, `src/prf_expand.rs`

line 107 @ d5e8426). The implementation computes the identical expression — PRFexpand keyed by the 32-byte little-endian nullifier over 0x84 followed by two 4-byte little-endian indices, reduced by ToBase (orchard-zsa, src/note.rs lines 479–489 @ cf801a5d). This derivation is why an issuance bundle cannot travel alone: requiring at least one Action Group both supplies $\mathsf{nf}_{0,0}$ and blocks replay — a transaction containing only an issuance bundle would have a SIGHASH computed solely from that bundle, so a duplicated bundle would share the SIGHASH (ZIP-227, *Rationale*).

In the builder, notes are created with ρ unset, via new_issue_note; once the transfer bundle's first nullifier is known, update_rho_for_issuance_note sets ρ and resamples rseed in a loop until the note commitment is not $\perp$[1] (orchard-zsa, src/note.rs lines 457–489 @ cf801a5d).

Reference notes. The first time an Asset is issued, the issuer MUST place a *reference note* as the first note of that Issue Action: a note of value $v = 0$ whose recipient $(\mathsf{d}, \mathsf{pk}_\mathsf{d})$ is the default diversified payment address (diversifier index 0) derived from the *all-zero* Orchard spending key (ZIP-227, *Reference Notes*). The implementation hard-codes the resulting 43-byte raw address as RAW_REFERENCE_RECIPIENT, byte-identical to the array in the ZIP, with ReferenceKeys::sk() = SpendingKey::from_bytes([0; 32]); the predicate is_reference_note requires zero value and exactly this recipient, get_reference_note accepts only the *first* note of an action, and the builder inserts the reference note itself when told the issuance is a first issuance — adding a recipient for an Asset that already has notes fails with CannotBeFirstIssuance (orchard-zsa, src/constants/reference_keys.rs lines 14–33 and src/issuance.rs lines 251–258, 459–507 @ cf801a5d). The reference note is retained in global state as $\mathsf{note}_\mathsf{ref}$ (§3.7).

The ZIP mandates reference notes and stores them in consensus state, but states no rationale for their existence. The evident function — that every issued Asset Base is guaranteed at least one note in the commitment tree, available as a ZIP-226 split-input source for a first spend — is our reading, stated nowhere in the ZIP text or its rationale sections; we classify the mechanism itself as *specified* and this purpose as *designed but unspecified*.

3.4 The issuance bundle and its encoding

An *Issuance Bundle* carries the issuer identifier, the list of Issue Actions, and one authorizing signature: issuer, which lets validators verify that each AssetId is properly associated with this issuer; vIssueActions, an array of Issue Actions; and issueAuthSig, the encoding of a signature over the transaction SIGHASH by the issuance authorizing key isk (ZIP-227, *Issuance Bundle*). An *Issuance Action* groups the notes issued for one Asset: the 32-byte assetDescHash, the note list, and a flags byte carrying finalize (ZIP-227, *Issuance Action*). Table 3 gives the v6 serialization; the encoding rigor caveat of the section introduction applies.

Three encoding facts deserve emphasis. First, the recipient field is exactly an Orchard raw payment address (the *Wallet Guide*, §"Raw encodings"), and together with the bundle's issuer and the action's assetDescHash, the four fields of an IssueNoteDescription determine the Issue Note completely (ZIP-230, *Issue Note Description*; ZIP-227, *Issue Note*) — the note commitment is deliberately absent, since validators recompute it (§3.3). Second, an action with nNotes = 0 is legal: it lets an issuer finalize an Asset without issuing more of it (ZIP-230, *Issuance Action Description*; finalization semantics in §3.8). Third, the flags byte is strict: IssuanceFlags::from_byte returns None whenever any bit other than the LSB is set, and deserialization fails on such bytes, enforcing ZIP-230's "remaining bits are set to 0" at parse time (orchard-zsa, src/issuance.rs lines 57–110; librustzcash-zsa, transaction/components/issuance.rs lines 89–91).

The librustzcash-zsa serializer pins down the absent-bundle corner cases: read_bundle reads issuer as a length-prefixed vector and returns None only for an empty issuer with zero actions, erroring on an empty issuer with non-empty actions and on a non-empty issuer with zero actions;

[1] Strictly, this conditions rseed on a negligible-probability event, a technical deviation from the ZIP's "sampled uniformly at random" (ZIP-227, *Issue Note*).

Field	Type	Notes
`issuerLength`	`compactSize`	MUST be 0 if `nIssueActions` = 0
`issuer`	`byte[issuerLength]`	= ik_encoding, 33 bytes when present
`nIssueActions`	`compactSize`	
`vIssueActions`	`IssueAction[nIssueActions]`	
`issueAuthSig`	`IssueAuthSignature`	present iff `nIssueActions` > 0
Per `IssueAction`:		
`assetDescHash`	`byte[32]`	
`nNotes`	`compactSize`	may be 0 (finalize-only)
`vNotes`	`IssueNoteDescription[nNotes]`	115 bytes each
`flagsIssuance`	`byte`	finalize at LSB; other bits MUST be 0
Per `IssueNoteDescription` (43 + 8 + 32 + 32 = 115 bytes):		
`recipient`	`byte[43]`	$\mathrm{LEBS2OSP}_{88}(\mathsf{d}) \,\|\|\, \mathsf{pk_d}^{\star}$
`value`	`uint64`	little-endian
`rho`	`byte[32]`	as for Orchard notes
`rseed`	`byte[32]`	as for Orchard notes

Table 3: ZSA issuance fields of the v6 transaction format (fork ZIP-230, *Transaction Format*, *Issuance Action Description*, and *Issue Note Description* @ zips-zsa `fd71419`; upstream text identical apart from the Withdrawn status).

`write_bundle` encodes an absent bundle as two zero `compactSizes`. The authorization serializes as a length-prefixed `sighashInfo` followed by the length-prefixed signature encoding, and the parser accepts exactly [`0x00`] as `sighashInfo` (`transaction/components/issuance.rs` lines 20–191 and `sighash_versioning.rs` @ `3f9b53fc`; the meaning of [`0x00`] is §3.5).

In the `orchard-zsa` object model, the type `IssueBundle<T: IssueAuth>` holds the validating key `ik` (an `IssueValidatingKey<ZSASchnorr>`), a non-empty vector of actions, and an authorization typestate `T`; an `IssueAction` holds the 32-byte `asset_desc_hash`, its `Vec<Note>`, and its `IssuanceFlags` (`src/issuance.rs` lines 47–124 @ `cf801a5d`).

3.5 Authorization: BIP-340 over the transaction sighash

The issuance authorization signature scheme IssueAuthSig is instantiated as a BIP-340 Schnorr signature over secp256k1 with the standard parameters — not RedPallas: issuance is a transparent capability, and the choice buys direct compatibility with existing secp256k1 tooling. Batch verification MAY be used, and precomputation MAY be used if and only if it produces equivalent results (ZIP-227, *Issuance Authorization Signature Scheme*). The associated types (same section):

$$\mathsf{Message} = \mathbb{B}^{\mathbb{Y}^{[\mathbb{N}]}}, \quad \mathsf{Signature} = \mathbb{B}^{\mathbb{Y}^{[64]}} \cup \{\bot\}, \quad \mathsf{Public} = \mathbb{B}^{\mathbb{Y}^{[32]}} \cup \{\bot\}, \quad \mathsf{Private} = \mathbb{B}^{\mathbb{Y}^{[32]}},$$

that is, arbitrary byte-sequence messages, 64-byte signatures, 32-byte x-only public keys, and 32-byte secret keys, with $\bot$ as the failure value. Signing fixes the BIP-340 auxiliary data to the all-zero block:

$$a := [\texttt{0x00}]^{32}, \qquad \sigma := \mathit{Sign}(\mathsf{isk}, M) \text{ with auxiliary data } a, \qquad \texttt{issueAuthSig} := [\texttt{0x00}] \,||\, \sigma,$$

returning $\bot$ on failure; validation returns 0 if $\sigma = \bot$, else 1 exactly when the BIP-340 $\mathit{Verify}(\mathsf{ik}, M, \sigma)$ succeeds[2] (ZIP-227, *Issuance Authorization Signing and Validation*). With the scheme byte, the two wire encodings are 33 bytes (ik_encoding) and 65 bytes (`issueAuthSig`).

What is signed. The message is the transaction SigHash as defined by protocol specification §4.10 under the ZIP-226 modifications, with sighash type SIGHASH_ALL, and the consensus rule

[2]The ZIP's Validate definition writes $\mathit{Verify}(\mathsf{key}, M, \sigma)$ for the parameter it names ik — an editorial slip we flag when citing (ZIP-227, *Issuance Authorization Signing and Validation*).

requires IssueAuthSig.Validate(ik, SigHash, `issueAuthSig`) = 1 (ZIP-227, *Specification: Consensus Rule Changes*). Under ZIP-246 the signature digest includes branch S.5, the issuance_digest, identical to the txid branch — so the issuance signature commits to *all* effecting data of the transaction, including the issuance bundle's own effecting data, though never to any signature (ZIP-246, *Signature Digest*). ZIP-246 also introduces sighash algorithm versioning: each bundle carries sighashInfo = [sighashVersion] || associatedData, versions are numbered from 0 per transaction version, version 0 is by convention the "commit to all effecting data" algorithm, and for v6 only version 0 is defined, with empty associated data for the Transparent, Sapling, Orchard, and Issuance bundles — hence the constant [`0x00`] the parser demands (ZIP-246, *Sighash versioning*).

Implementation. The `ZSASchnorr` scheme wraps the `secp256k1` crate: signing builds a keypair from isk and calls `sign_schnorr_with_aux_rand(msg, keypair, &[0u8; 32])` — the ZIP's $a = [\texttt{0x00}]^{32}$ — ik is the x-only public key, and `encode()` of key and signature each prepend `ALGORITHM_BYTE = 0x00`, with `decode()` rejecting any other lead byte (`orchard-zsa`, `src/issuance/auth.rs` lines 163–303 @ `cf801a5d`). Bundle construction is a typestate chain — `AwaitingNullifier` → (`update_rho`) → `AwaitingSighash` → (`prepare(sighash)`) → `Prepared` → (`sign(isk)`) → `Signed` — so a bundle cannot be signed before its notes' ρ values and the sighash exist (`src/issuance.rs` lines 261–304 and 404–599 @ `cf801a5d`). On the transaction side, `v6_signature_hash` includes the txid issuance_digest, and the builder signs the issue bundle with `prepare(*shielded_sig_commitment)` where that commitment is `signature_hash(tx, SignableInput::Shielded, txid_parts)` — the same SIGHASH_ALL shielded digest the Sapling and Orchard signatures sign (`librustzcash-zsa`, `transaction/sighash_v6.rs` lines 10–51 and `transaction/builder.rs` lines 1416–1417, 1453–1460 @ `3f9b53fc`). The builder also realizes the $\mathsf{nf}_{0,0}$ rule constructively: `first_nullifier` returns the first Action's nullifier only for an OrchardZSA bundle, and building with an issuance builder but no such bundle fails with `BundleTypeNotSatisfiable` (`transaction/builder.rs` lines 1356–1366 and 1594–1602 @ `3f9b53fc`).

A divergence to flag. ZIP-246's txid and signature digests include a sixth branch, T.6/S.6 memo_digest (ZIP-246, *TxId Digest* and *Signature Digest*), but `librustzcash-zsa`'s `to_hash` hashes only the header, transparent, Sapling, Orchard, and issuance branches — the fork does not implement ZIP-231 memo bundles (`transaction/txid.rs` lines 415–460 @ `3f9b53fc`). The digest the fork code signs is therefore not byte-identical to the digest ZIP-246 specifies; whichever is corrected, the issuance signature's coverage claim above should be read against the fork's five-branch digest today.

3.6 Transaction digests

The v6 digest tree extends the ZIP-244 structure the *Ironwood Guide* presents in §"Conservation without disclosure: value commitments and the binding signature": the txid digest is BLAKE2b-256 with personalization `ZcashTxHash_` || CONSENSUS_BRANCH_ID over six branches T.1–T.6 (header, transparent, Sapling, Orchard, issuance, memo), and the signature digest has the same six-branch structure with S.5 identical to T.5; a signature digest is produced for each transparent input, Sapling input, OrchardZSA Action, and additionally for each Issuance Action (ZIP-246, *TxId Digest* and *Signature Digest*). The issuance branches are three nested hashes and one authorizing hash, each over

the wire encodings of Table 3 (ZIP-246, *txid_digest* T.5 and *A.4: issuance_auth_digest*):[3]

$$\begin{aligned}
\text{issuance_digest} &= \text{BLAKE2b-256}\big(\texttt{ZTxIdSAIssueHash},\\
&\qquad \texttt{issuerLength} \,\|\, \texttt{issuer} \,\|\, \text{issue_actions_digest}\big),\\
\text{issue_actions_digest} &= \text{BLAKE2b-256}\big(\texttt{ZTxIdIssuActHash},\\
&\qquad \|_{\text{actions}}\ \texttt{assetDescHash} \,\|\, \text{issue_notes_digest} \,\|\, \texttt{flagsIssuance}\big),\\
\text{issue_notes_digest} &= \text{BLAKE2b-256}\big(\texttt{ZTxIdIAcNoteHash},\\
&\qquad \|_{\text{notes}}\ \texttt{recipient} \,\|\, \texttt{value} \,\|\, \texttt{rho} \,\|\, \texttt{rseed}\big),\\
\text{issuance_auth_digest} &= \text{BLAKE2b-256}\big(\texttt{ZTxAuthZSAOrHash},\ \texttt{issueAuthSig}\ \text{field encoding}\big),
\end{aligned}$$

where the A.4 preimage is the ZIP-230 `IssueAuthSignature` composite

$$\texttt{sizeSighashInfo} \,\|\, \texttt{sighashInfo} \,\|\, \texttt{sizeSignature} \,\|\, \texttt{signature}$$

(ZIP-230, *Issuance Authorization Signature*), and a transaction without issuance components hashes the empty string under the same personalizations.

The code computes exactly these. On the txid side, `hash_issue_bundle_txid_data` hashes

$$\text{compactSize}(|\text{ik\_encoding}|) \,\|\, \text{ik\_encoding} \,\|\, \text{issue\_actions\_digest}$$

under `ZTxIdSAIssueHash`; the actions digest hashes, per action, the 32-byte hash, the notes digest, and the flags byte; the notes digest hashes, per note, the 43-byte recipient, the 8-byte little-endian value, and the two 32-byte fields. On the authorizing side, `hash_issue_bundle_auth_data` hashes

$$\text{compactSize}(|\text{sighashInfo}|) \,\|\, \text{sighashInfo} \,\|\, \text{compactSize}(65) \,\|\, [\texttt{0x00}] \,\|\, \sigma,$$

asserting the 65-byte length and the `0x00` lead. The empty-bundle variants hash the empty string, and all four personalizations match ZIP-246; both digests live in `src/bundle/commitments/issuance.rs` lines 11–86 (`orchard-zsa` @ `cf801a5d`). For the wiring into transaction identifiers, `TxIdDigester::digest_issue` maps the bundle to its `commitment()`; `to_hash` appends the digest only when the version `has_orchard_zsa()`; and `BlockTxCommitmentDigester` takes the authorizing commitment (`librustzcash-zsa`, `transaction/txid.rs` lines 381–383, 445–452, 595–598 @ `3f9b53fc`).

Deterministic regression digests are pinned in the tests — for a seeded two-asset bundle,

$$\begin{aligned}
\text{issuance_digest} &= \texttt{ee70e3b61674fd0428ac0020cc4fc5819386e39c4eb3c63357c84c998195bcdb},\\
\text{issuance_auth_digest} &= \texttt{6df77af7b5323d99376336b770e4c5b06ffc195de81ac7692d1b08b6eb19534d}
\end{aligned}$$

— and cross-implementation test vectors ship in `src/test_vectors/` (`asset_base.rs`, `issuance_auth_sig.rs`), exercised by the asset-base and authorization tests (`orchard-zsa`, `src/bundle/commitments/issuance.rs` lines 156–190 and `src/test_vectors/` @ `cf801a5d`). The upstream ZIP-227 Test Vectors section, by contrast, still reads LINK TBD.

3.7 Global issuance state and the consensus rules

Supply integrity is the one property issuance cannot delegate to cryptography: no transaction field simultaneously witnesses everything issued and everything burnt for an Asset, so — along the lines of the ZIP-209 pool turnstile — every node maintains a per-Asset circulation record (ZIP-227, *Rationale for Global Issuance State*).

[3] Two editorial defects in ZIP-246 to note when citing: the T.5 child list labels `issuerLength/issuer/issue_actions_digest` as T.5a/T.5b/T.5c, while the following section headings call `issue_actions_digest` "T.5a" and `issue_notes_digest` "T.5a.i"; and the A.4 empty case reads "In the case that the transaction has no Orchard Actions" where "no Issuance Actions" is evidently intended.

Definition 3.2 (Global issuance state)**.** The map

$$\begin{aligned}\mathsf{issued\_assets} : \mathcal{E}(\mathbb{F}_{p_{\mathsf{Pallas}}})^* &\to \{0\,..\,\mathsf{MAX\_ISSUE}\} \times \{0,1\} \times \mathsf{Note}^{\mathsf{Issue}},\\ \mathsf{AssetBase} &\mapsto (\mathsf{balance}, \mathsf{final}, \mathsf{note}_{\mathsf{ref}}),\end{aligned}$$

records, for every issued Asset, the amount in circulation (issued minus burnt), the finalization status — which can never change from 1 to 0 — and the Asset's reference note. A missing entry ($\mathsf{issued\_assets}(\mathsf{AssetBase}) = \bot$) is read as $\mathsf{balance} = 0$, $\mathsf{final} = 0$, $\mathsf{note}_{\mathsf{ref}} = \bot$; the constant $\mathsf{MAX\_ISSUE} = 2^{64} - 1$ caps the total supply of any Custom Asset (ZIP-227, *Global Issuance State*).

The map threads through the chain deterministically: it MUST be updated by a node during the processing of any transaction that contains burn information or an issuance bundle; the input state for the OrchardZSA activation block is the empty map; the input state of a block's first transaction is the final state of the preceding block; each subsequent transaction's input state is the preceding transaction's output state; and the block's final state is its last transaction's output state (ZIP-227, *Management of the Global Issuance State*).

Per-transaction rules. For a v6 transaction (ZIP-227, *Specification: Consensus Rule Changes*):

(T1) `nActionGroupsOrchard` MUST be 0 or 1, and `nAGExpiryHeight` MUST be 0; the v6 parser of `librustzcash-zsa` enforces both at deserialization, in `transaction/components/orchard.rs` lines 107–132 @ `3f9b53fc`;

(T2) the output state $\mathsf{issued\_assets}_{\mathsf{OUT}}$ MUST be initialized to the input state $\mathsf{issued\_assets}_{\mathsf{IN}}$;

(T3) the assetBurn set MUST satisfy the ZIP-226 rules (§4: tuples $(\mathsf{AssetBase}, \mathsf{v})$ with the Native Asset excluded, $0 < \mathsf{v} \le \mathsf{MAX\_BURN\_VALUE} = 2^{63} - 1$, and no duplicate AssetBase; ZIP-226, burn rules); and

(T4) for every $(\mathsf{AssetBase}, \mathsf{v}) \in \mathsf{assetBurn}$, the node MUST check $\mathsf{balance} \ge \mathsf{v}$ for the entry of AssetBase in $\mathsf{issued\_assets}_{\mathsf{OUT}}$ and subtract v from that balance, *prior to* processing the issuance bundle.

Issuance-bundle rules. For a transaction carrying an issuance bundle (ZIP-227, *Specification: Consensus Rule Changes*):

(I1) the transaction MUST also contain at least one OrchardZSA Action Group (the ρ/replay rationale of §3.3);

(I2) the encoding ik_encoding MUST be taken from the `issuer` field and MUST start with the byte `0x00` (BIP-340);

(I3) the signature MUST satisfy $\mathsf{IssueAuthSig.Validate}(\mathsf{ik}, \mathsf{SigHash}, \texttt{issueAuthSig}) = 1$;

(I4) for each Issue Action: every `IssueNoteDescription` MUST be a valid ZIP-230 encoding; the AssetBase is derived from assetDescHash and issuer (§3.2); and the Asset's final bit in $\mathsf{issued\_assets}_{\mathsf{OUT}}$ MUST NOT be 1;

(I5) if $\mathsf{note}_{\mathsf{ref}} = \bot$, the first Issue Note MUST be a reference note (§3.3), and the node MUST set $\mathsf{note}_{\mathsf{ref}}$ to it;

(I6) for every Issue Note note_j: its ρ MUST equal the value computed per §3.3; the supply update

$$\mathsf{issued\_assets}_{\mathsf{OUT}}.\mathsf{balance} + \mathsf{v}_j \le \mathsf{MAX\_ISSUE}, \qquad \mathsf{issued\_assets}_{\mathsf{OUT}}.\mathsf{balance} \mathrel{+}= \mathsf{v}_j$$

MUST hold and be applied; and the node MUST compute the note commitment per ZIP-226;

(I7) if finalize = 1 in `flagsIssuance`, the node MUST set final = 1.

The commitments enter the tree in a fixed order: for each Action Group of the OrchardZSA bundle, the commitment of every new note per Action; then, for each Issue Action of the issuance bundle, the commitment of every Issue Note (ZIP-227, *Addition to the Note Commitment Tree*).

The rationale ties the pieces together: balances of issued Assets must never go negative, MAX_ISSUE is a practical limit that also lets an issuer create an Asset's complete supply in a single transaction, and the same map records finalization so that further issuance of a finalized Asset is rejected (ZIP-227, *Rationale for Global Issuance State*). Beyond consensus, the ZIP RECOMMENDS that full validators keep a mutable `issuanceSupplyInfoMap` from AssetId to (`totalSupply`, `finalize`) for wallets and applications tracking total supply (ZIP-227, *Implementing Zcash Nodes*).

The library realization. Function `verify_issue_bundle` takes the bundle, the sighash, a caller-supplied view of global state (`get_global_records`), and the first nullifier. It rejects any sighash kind other than `AllEffecting`, verifies the BIP-340 signature over the 32-byte sighash with the bundle's ik, folds the actions over a `BTreeMap<AssetBase, AssetRecord>`, and returns the updated records — `amount`, `is_finalized`, `reference_note` — for the caller to persist (`orchard-zsa`, `src/issuance.rs` lines 647–802 @ `cf801a5d`). The fold realizes the per-action and per-note rules as typed errors:

- `IncorrectRhoDerivation` — a note's ρ differs from the recomputed derivation of §3.3;
- `MissingReferenceNoteOnFirstIssuance` — no record exists for the Asset, and the action's first note is not a reference note;
- `IssueActionPreviouslyFinalizedAssetBase` — reissuance against a finalized record;
- `IssueBundleIkMismatchAssetBase` and `AssetBaseCannotBeIdentityPoint` — a note's Asset Base differs from the one `IssueAction::verify` derives from (ik, assetDescHash), or the derived point is the Pallas identity;
- `ValueOverflow` — the checked `u64` value sum overflows, the code's realization of MAX_ISSUE $= 2^{64} - 1$ (`src/issuance.rs` lines 216–244; `src/value.rs` lines 143–145).

A signature-skipping variant, `check_issue_bundle_without_sighash`, exists for verifying historical blocks from a trusted checkpoint (`src/issuance.rs` lines 647–705 @ `cf801a5d`).

Three boundary observations, each load-bearing for anyone re-deriving consensus from these libraries:

- *Empty-action divergence.* The implementation rejects an action with no notes unless it is finalized (`IssueActionWithoutNoteNotFinalized`, `src/issuance.rs` lines 216–219), but ZIP-227's consensus-rule list contains no such MUST, and ZIP-230 merely notes that `nNotes` may be zero "to only finalize". A ZIP-conformant validator would accept an empty, non-finalizing action that the reference implementation rejects.
- *First-issuance detection.* The ZIP keys the reference-note requirement on $\mathsf{note}_{\mathsf{ref}} = \bot$; the code keys it on the complete absence of an `AssetRecord` (`get_global_records` returning `None`). The two are equivalent under the invariant that $\mathsf{note}_{\mathsf{ref}}$ is set exactly on first issuance — rule (I5) — and no rule ever clears it, so an entry exists if and only if its $\mathsf{note}_{\mathsf{ref}}$ is set.
- *Node-side enforcement locus.* Neither `orchard-zsa` nor `librustzcash-zsa` contains the issued_assets chain-state maintenance or the consensus caller of `verify_issue_bundle`: the libraries return updated records for a node to persist, and the QEDit node integration is not among our local ground-truth repositories. The global-state rules are therefore *specified and library-supported*, with their node enforcement unverified locally.

3.8 Finalization

The finalize boolean signals that no further issuance of the specific Custom Asset is possible: every issuance action carries it in `flagsIssuance` (a requirement of the ZIP's Requirements section), setting it sets the Asset's final bit — rule (I7) — and transactions attempting to issue further amounts of a previously finalized Asset are rejected — rule (I4) (ZIP-227, *Requirements* and *Issuance Action*). The bit is monotone: consensus state never changes final from 1 to 0 (Definition 3.2).

Finalization is the ZIP's whole lifecycle policy, and its Concrete Applications read as patterns over one bit (ZIP-227, *Concrete Applications*):

- *One-time issuance*: set finalize = 1 on the first action; the entire supply exists from one transaction — MAX_ISSUE is sized to permit this (§3.7).
- *Stopping supply*: issue with finalize = 1 in a final issuance transaction, or in one containing a single note of value 0 (the wire format admits note-less finalizing actions as well, Table 3; the library insists on the finalizing variant, §3.7).
- *NFTs*: issue multiple actions in one transaction, each with value = 1 at the Asset's fundamental unit and finalize = 1 — each action's Asset is thereafter unique and non-replicable.

Finalization is also the entirety of the compromise story. No key rotation exists for issuance keys; on compromise of isk, the ZIP's guidance is to set finalize = 1 for all Assets issued with that key, switch to a new isk — hence a new issuer identifier, since issuer = ik_encoding — and issue new Assets with new AssetIds (ZIP-227, *Issuance Key Compromise*). The old Assets keep circulating and burning; they simply never grow again.

3.9 Fees

The fee treatment of issuance is the sharpest fork/upstream split in the ZSA stack, and we bin it explicitly. The QEDit fork of ZIP-317 adds two contributions to the logical-action count of the conventional fee (the *Wallet Guide*, §"Fees (ZIP-317)", for the surrounding machinery: *marginal_fee* = 5000 zatoshis, *grace_actions* = 2):

$$\begin{aligned} \mathit{contribution}_{\mathsf{ZSAIssuance}} &= \mathit{nZSAIssueNotes}, \\ \mathit{contribution}_{\mathsf{ZSACreation}} &= \mathit{creation\_cost} \cdot \mathit{nAssetCreations}, \end{aligned}$$

where *nZSAIssueNotes* counts `IssueNote` outputs, *nAssetCreations* counts Custom Assets newly added to the global issuance state, and *creation_cost* = 100 logical actions (fork ZIP-317, *Fee calculation* @ zips-zsa `fd71419`). The stated rationale: every new Asset adds a row that validators track in perpetuity (§3.7), and the two-orders-of-magnitude creation cost disincentivizes junk assets.

Upstream, these contributions no longer exist: ZIP-317's change history records, on 2026-06-26, "Removed the ZSA issuance and asset-creation fee contributions (ZIP 227)" (ZIP-317, change history @ zips `6961098`) — while ZIP-227's own *Changes to ZIP 317* section still claims the conventional fee accounts for both. The formulas above survive only in the QEDit fork and its libraries: *fork-specified*, implemented in `librustzcash-zsa` behind `cfg(zcash_unstable = "nu7")` (`transaction/fees/zip317.rs`, `CREATION_COST = 100` @ `3f9b53fc`), and awaiting an upstream resolution that the NU7 process (§1) has yet to provide.

4 Transfer and burn (ZIP-226)

ZIP-226 (*Transfer and Burn of Zcash Shielded Assets*; Status: Draft; owners Pablo Kogan and Vivek Arte of QEDit, Daira-Emma Hopwood, and Jack Grigg) defines, jointly with the issuance protocol of ZIP-227 (§3), the OrchardZSA transfer and burn protocol. We develop it as four deltas against

the Orchard baseline, each citing the construction it modifies in the *Ironwood Guide* rather than re-deriving it: the note gains one field, the *Asset Base* (§4.1); the note commitment becomes a case split that keeps ZEC commitments identical to NU5 (§4.2); the value commitment trades its fixed value base for the per-asset base, which turns the binding signature into a per-asset balance check and gives burn its consensus meaning (§4.3, §4.4); and dummy spends give way to *Split Inputs* for Custom Assets (§5). We then state the extended Action statement (§4.6) and the carrier format with its digests and fees (§4.7).

Ground truth for this section: upstream `zips` at commit `69610984`, the QEDit fork `zips-zsa` at `fd71419a`, crate `orchard-zsa` at `cf801a5d` (branch `zsa1`), and `librustzcash-zsa` at `3f9b53fc` (branch `zsa1`); we verified every cited artifact at these commits. ZIP-226 names the QED-it `zcash-test-vectors` and the QED-it forks of `zcashd`, `orchard`, `librustzcash`, and `halo2` as its test vectors and reference implementations. Deployment standing is fixed once, in §1, and not relitigated per object; two upstream facts nonetheless shape the text. First, ZIP-226 states that ZSA "is scheduled to be deployed in Network Upgrade 7 (NU7)" and "currently assumes that this will be the case", while the transaction format it rides on has been withdrawn upstream (§4.7). Second, upstream ZIP-226 is now defined relative to the protocol with ZIP-2005 (Ironwood Quantum Recoverability) applied, which is expected to deploy before any ZSA activation; the QEDit fork's ZIP-226 lacks this clause, and the pinned implementation does not implement ZIP-2005 constructs. Where the difference is material — the split-note ψ^{nf} (§5) and the lead byte (§4.7) — we flag it in place.

4.1 One new field: the OrchardZSA note

Definition 4.1 (OrchardZSA note)**.** An OrchardZSA note extends the Orchard note — the tuple $(\mathsf{addr}, v, \rho, \psi, \mathsf{rcm})$ of the *Ironwood Guide*, "The note and its commitment" — by exactly one field:

$$\mathbf{n} := (\mathsf{addr},\ v,\ \mathsf{AssetBase},\ \rho,\ \psi,\ \mathsf{rcm}),$$

where $\mathsf{AssetBase} \in \mathcal{E}^* := \mathcal{E}(\mathbb{F}_{p_{\mathsf{Pallas}}}) \setminus \{\mathcal{O}\}$ is "a valid group element that is not the identity and is not bottom" (ZIP-226; the ZIP writes $\mathbb{P}^*$ for $\mathcal{E}^*$). All other components are as in protocol specification §3.2. Formally,

$$\begin{aligned}\mathsf{Note}^{\mathsf{OrchardZSA}} := {} & \{0,1\}^{88} \times \mathsf{KA}^{\mathsf{Orchard}}.\mathsf{Public} \times \{0,\dots,2^{64}-1\} \times \mathcal{E}^* \\ & \times \mathbb{F}_{p_{\mathsf{Pallas}}} \times \mathbb{F}_{p_{\mathsf{Pallas}}} \times \mathsf{NoteCommit}^{\mathsf{Orchard}}.\mathsf{Trapdoor},\end{aligned}$$

where the ZIP's $\mathbb{F}_{q_P}$ is this series' $\mathbb{F}_{p_{\mathsf{Pallas}}}$ and $\ell_{\mathsf{value}} = 64$ (ZIP-226).

The byte encoding of the new field is the star-encoding of the series' conventions: ZIP-226 sets $\mathsf{asset\_base} := \mathrm{LEBS2OSP}_{\ell_P}(\mathrm{repr}_{\mathbb{P}}(\mathsf{AssetBase}))$, which is $\mathsf{AssetBase}^\star \in \{0,1\}^{256}$ in 32 bytes.

An Asset Base is never a free choice of the transactor. Every Custom Asset's base is derived by the ZIP-227 issuance chain — constructed in §3, reproduced here because the soundness argument of §5 depends on its shape:

$$\begin{aligned}\mathsf{assetDescHash} &:= \text{BLAKE2b-256}\big(\texttt{"ZSA-AssetDescCRH"},\ \mathsf{asset\_desc}\big),\\ \mathsf{AssetId} &:= (\mathsf{issuer},\ \mathsf{assetDescHash}),\\ \mathsf{EncodeAssetId}(\mathsf{AssetId}) &:= \texttt{0x00} \,\|\, \mathsf{issuer} \,\|\, \mathsf{assetDescHash},\\ \mathsf{AssetDigest} &:= \text{BLAKE2b-512}\big(\texttt{"ZSA-Asset-Digest"},\ \mathsf{EncodeAssetId}(\mathsf{AssetId})\big),\\ \mathsf{AssetBase} &:= \mathsf{GroupHash}^{\mathsf{Pallas}}\big(\texttt{z.cash : OrchardZSA},\ \mathsf{AssetDigest}\big)\end{aligned}$$

(ZIP-227, which names the last map ZSAValueBase). For the native Asset (ZEC/TAZ) the Asset Base is instead *defined* to be

$$V^{\mathsf{Orchard}} := \mathsf{GroupHash}^{\mathsf{Pallas}}(\texttt{z.cash : Orchard-cv}, \texttt{"v"}),$$

the fixed value-commitment base of the *Ironwood Guide*, "Conservation without disclosure: value commitments and the binding signature" (ZIP-226). This single identification is what keeps every ZEC computation below identical to NU5: ZIP-226 uses $\mathsf{ValueCommit}^{\mathsf{Orchard}}_{\mathsf{rcv}}(v)$ as $\mathsf{ValueCommit}^{\mathsf{OrchardZSA}}_{\mathsf{rcv}}(V^{\mathsf{Orchard}}, v)$. The implementation realizes it as `AssetBase::zatoshi() = hash_to_curve("z.cash:Orchard-cv")(b"v")` (orchard-zsa, `src/note/asset_base.rs`, `src/constants/fixed_bases.rs`).

4.2 The note commitment: a case split

Definition 4.2 (OrchardZSA note commitment)**.** For an OrchardZSA note,

$$\mathsf{NoteCommit}^{\mathsf{OrchardZSA}}_{\mathsf{rcm}}(\mathsf{g_d}^\star, \mathsf{pk_d}^\star, v, \rho, \psi, \mathsf{AssetBase}) := \begin{cases} \mathsf{NoteCommit}^{\mathsf{Orchard}}_{\mathsf{rcm}}(\mathsf{g_d}^\star, \mathsf{pk_d}^\star, v, \rho, \psi) & \text{if } \mathsf{AssetBase} = V^{\mathsf{Orchard}}, \\ \mathsf{cm}_{\mathsf{ZSA}} & \text{otherwise,} \end{cases}$$

where

$$\mathsf{cm}_{\mathsf{ZSA}} := \mathsf{Sinsemilla}_{\texttt{z.cash:ZSA-NoteCommit-M}}\big(\mathsf{g_d}^\star \,\|\, \mathsf{pk_d}^\star \,\|\, \mathrm{LE}_{64}(v) \,\|\, \mathrm{LE}_{255}(\rho) \,\|\, \mathrm{LE}_{255}(\psi) \,\|\, \mathsf{AssetBase}^\star\big) \dotplus [\mathsf{rcm}]\,\mathsf{GroupHash}^{\mathsf{Pallas}}\big(\texttt{z.cash : Orchard-NoteCommit-r, ""}\big)$$

(ZIP-226; the ZIP writes I2LEBSP for the series' LE_k). The signature is

$$\mathsf{NoteCommit}^{\mathsf{OrchardZSA}} : \mathsf{NoteCommit}^{\mathsf{Orchard}}.\mathsf{Trapdoor} \times \{0,1\}^{256} \times \{0,1\}^{256} \times \{0,\ldots,2^{64}-1\} \times \mathbb{F}_{p_{\mathsf{Pallas}}} \times \mathbb{F}_{p_{\mathsf{Pallas}}} \times \mathcal{E}^* \to \mathsf{NoteCommit}^{\mathsf{Orchard}}.\mathsf{Output},$$

with trapdoor and output types those of $\mathsf{NoteCommit}^{\mathsf{Orchard}}$, unchanged (ZIP-226).

The two branches carry the design's two promises. In the native branch the Asset Base is not hashed at all: a ZEC note commitment under OrchardZSA *is* the NU5 commitment, bit for bit. In the ZSA branch the message appends $\mathsf{AssetBase}^\star$ (256 bits) after ψ, against the baseline layout of the *Ironwood Guide*, "The note and its commitment":

$$\mathsf{g_d}^\star \,\|\, \mathsf{pk_d}^\star \,\|\, \mathrm{LE}_{64}(v) \,\|\, \mathrm{LE}_{255}(\rho) \,\|\, \mathrm{LE}_{255}(\psi) \longmapsto \cdots \,\|\, \mathrm{LE}_{255}(\psi) \,\|\, \mathsf{AssetBase}^\star,$$

under the fresh hash domain `z.cash:ZSA-NoteCommit-M` but the *same* randomness base $\mathsf{GroupHash}^{\mathsf{Pallas}}$(`z.cash : Orchard-NoteCommit-r, ""`). ZIP-226's rationale: "the nested structure of the Sinsemilla-based commitment allows us to add the Asset Base as a final recursive step", and the output "is still indistinguishable from the original Orchard ZEC note commitments, by definition of the Sinsemilla hash function". Because the trapdoor and output types are unchanged, ZSA commitments land in the same note commitment tree as ZEC commitments and are indistinguishable from them there (ZIP-226). One caveat bounds the claim: Orchard (v5) note commitments *are* distinguishable from OrchardZSA (v6) note commitments, because the two protocols use different transaction versions (ZIP-226).

Remark 4.3 (Implementation)**.** Function `NoteCommitment::derive` computes *both* commitments. For ZEC it calls `CommitDomain::new` on the prefix `"z.cash:Orchard-NoteCommit"`; for ZSA it calls `new_with_separate_domains` on the *M*-prefix `"z.cash:ZSA-NoteCommit"` and the shared *r*-prefix `"z.cash:Orchard-NoteCommit"`. It then selects between them in constant time on `asset.is_zatoshi()` (orchard-zsa, `src/note/commitment.rs`). The sinsemilla crate appends the `-M` and `-r` suffixes to these prefixes (sinsemilla 0.1.0, `src/lib.rs`).

The nullifier of an ordinary (non-split) OrchardZSA note is generated exactly as in Orchard (protocol specification §4.16; *Ironwood Guide*, "The nullifier, and the loop back to minting"); only Split Inputs use the modified, randomized formula of §5 (ZIP-226).

4.3 Per-asset value: the variable-base value commitment

Definition 4.4 (OrchardZSA net value commitment)**.** Each Action commits to its net value change in its Asset:

$$\mathsf{cv}^{\mathsf{net}} := \mathsf{ValueCommit}^{\mathsf{OrchardZSA}}_{\mathsf{rcv}}(\mathsf{AssetBase}, v^{\mathsf{net}}) = [\,v^{\mathsf{net}}\,]\,\mathsf{AssetBase} + [\mathsf{rcv}]\,R^{\mathsf{Orchard}},$$
$$v^{\mathsf{net}} := v_{\mathrm{old}} - v_{\mathrm{new}}, \qquad R^{\mathsf{Orchard}} := \mathsf{GroupHash}^{\mathsf{Pallas}}(\texttt{z.cash : Orchard-cv}, \texttt{"r"}),$$

where AssetBase is the Action's Asset Base and the randomness base R^{Orchard} is unchanged (ZIP-226).

Exactly one thing changes against the baseline $\mathsf{cv}^{\mathsf{net}} = [v_{\mathrm{old}} - v_{\mathrm{new}}]\,V^{\mathsf{Orchard}} + [\mathsf{rcv}]\,R^{\mathsf{Orchard}}$ of the *Ironwood Guide*, "Conservation without disclosure: value commitments and the binding signature": the fixed-base multiplication by V^{Orchard} becomes a variable-base multiplication by the Action's AssetBase, while the randomness term keeps its fixed base (ZIP-226). For ZEC, $\mathsf{AssetBase} = V^{\mathsf{Orchard}}$ and the commitment reduces to the NU5 one (§4.1).

The prover still signs the SIGHASH with $\mathsf{bsk} = \sum_{i=1}^{n} \mathsf{rcv}_i$ over the n Actions of a transfer (ZIP-226); the verifier's key acquires burn terms — one zero-trapdoor commitment subtracted per assetBurn entry, each a bare multiple of its base — displayed in this volume as (2) (§5.1). Why this checks balance *per asset*: "the committed values must sum to zero per Asset Base, as the Pedersen commitments add up homomorphically only with respect to the same value base point" (ZIP-226). Partitioning the Actions into a ZEC set and a Custom Asset set, the custom-asset commitments must cancel against the declared burns up to pure randomness terms — the cancellation equation is displayed in §5.2 — so $\mathsf{bvk} = [\mathsf{bsk}]\,R^{\mathsf{Orchard}}$ holds if and only if, per Custom Asset, the sum of net Action values equals its assetBurn entry (or 0 if absent), and the ZEC component equals $v^{\mathsf{Orchard}}_{\mathrm{balance}}$ (ZIP-226). The binding-signature mechanics — bsk/bvk cancellation and RedPallas on the base R^{Orchard} — are those of the *Ironwood Guide*, "Conservation without disclosure: value commitments and the binding signature", and we do not restate them.

Remark 4.5 (Implementation: derive_bvk)**.** Function `derive_bvk_raw` computes the ZIP equation verbatim: $\sum \mathsf{cv}^{\mathsf{net}}$ minus the zero-trapdoor commitment of `value_balance` at `AssetBase::zatoshi()` minus, over the burn set, the zero-trapdoor commitment of each `value` at its `asset` (orchard-zsa, `src/bundle.rs`). Function `ValueCommitment::derive` computes $V \cdot \mathsf{value} + R \cdot \mathsf{rcv}$ with V the point `asset.cv_base()` and R the point `hash_to_curve("z.cash:Orchard-cv")` applied to `b"r"`, the value signed per the sign of the value sum (orchard-zsa, `src/value.rs`).

4.4 Burn

Burning is the value-balance mechanism read as an exit door. The mechanism "does NOT send Assets to a non-spendable address, it simply reduces the total number of units of a given Custom Asset in circulation"; it is enforced at consensus level as an extension of the value-balance mechanism, and it "makes it globally provable that a given amount of a Custom Asset has been destroyed". The Native Asset (ZEC/TAZ) cannot be burnt this way (ZIP-226).

ZIP-226 attaches three consensus rules to the burn set, whose cardinality it denotes L:

1. for every $(\mathsf{AssetBase}, v) \in \mathsf{assetBurn}$: $\mathsf{AssetBase} \neq V^{\mathsf{Orchard}}$;
2. $v > 0$ and $v \leq \mathsf{MAX\_BURN\_VALUE} := 2^{63} - 1$;[4]
3. every AssetBase has at most one entry in assetBurn.

[4] ZIP-230's AssetBurn table instead says valueBurn is "checked by consensus to be non-zero, and less than MAX_BURN_VALUE". The implementation rejects only amounts strictly greater than MAX_BURN_VALUE, i.e. permits equality, agreeing with ZIP-226 (orchard-zsa, `src/bundle/burn_validation.rs`, which defines $\mathsf{MAX\_BURN\_VALUE} = (1 \ll 63) - 1$). A specification-text defect, not a consensus divergence.

Burn data lives *per Action Group*: the ActionGroupDescription carries nAssetBurn (compactSize) and vAssetBurn, an array of 40-byte entries — asset_base (byte[32]) followed by valueBurn (uint64, little-endian) — so that parties assembling separate Action Groups can each burn their own Assets (withdrawn ZIP-230; §4.7).

The transaction-level field valueBalanceOrchard (int64) remains the net *ZEC-only* flow, "the net value of Orchard spends minus outputs" (withdrawn ZIP-230), and the transparent protocol is unchanged, so a Custom Asset cannot be unshielded by transfer: "All Custom Assets are contained within the shielded pool" (ZIP-226). Burning is the only exit. What a burn reveals is exactly its assetBurn entry: "the amount and identifier of the Asset being burnt". What transfers hide: "the overall balance is checked without revealing which (or how many distinct) Assets are being transferred"; OrchardZSA ZEC commitments are indistinguishable from non-native OrchardZSA commitments, and the amounts and identifiers of transferred Assets stay private (ZIP-226).

A burn also updates ZIP-227's global issuance state: issued_assets(AssetBase).balance $\in \{0, \ldots, \mathsf{MAX\_ISSUE}\}$ is "computed as the amount of the Asset that has been issued less the amount of the Asset that has been burnt", and the map MUST be updated during processing of any transaction containing burn information (ZIP-227). Supply sufficiency — that a burn not exceed the circulating supply — is therefore a state-level rule of the issuance protocol, not one of ZIP-226's three per-transaction rules above; we construct the state machine in §3. The implementation enforces both layers side by side: `validate_bundle_burn` returns the per-transaction errors `ZatoshiAsset`, `ZeroAmount`, `InvalidAmount`, and `DuplicateAsset`, and the state-level errors `AssetNotFoundInState` and `InsufficientSupply` (orchard-zsa, `src/bundle/burn_validation.rs`).

4.5 Split Inputs

Padding is where the asset dimension bites. An Orchard transfer with more outputs than inputs pads with *dummy* spends: zero-value notes whose Merkle membership check is gated off by v_{old} (constraint A3 of the *Ironwood Guide*, "The Action statement, formally"). For Custom Assets that padding is unsound, because the circuit forces the output note's Asset Base to equal the input note's (§4.6) — and a dummy input's Asset Base is chosen freely by the prover. ZIP-226 states the attack that must be excluded: without enforcing that every Custom Asset input exists in the note commitment tree, "the prover could input a multiple (or linear combination) of an existing Asset Base, and thereby attack the network by overflowing the ZEC value balance and hence counterfeiting ZEC funds". The fix: dummy notes remain in use for ZEC notes only; Custom Assets always prove Merkle membership, padding instead with Split Inputs, which force the output's Asset Base to equal a genuinely issued, GroupHash-derived base (ZIP-226).

The construction is developed once, in §5: the *Split Input* — a copy of a previously issued, tree-resident note of the target Asset, spent with split_flag = 1 and its value zeroed in the value-commitment computation only (Definition 5.1); its randomized nullifier with the offset point L^{Orchard}, (3); the three-way divergence over the derivation of ψ^{nf} (§5.6); and the builder mechanics of padding per asset, dummy notes for ZEC and split spends for Custom Assets (§5.8). The three wallet strategies ZIP-226 offers for sourcing the copied note are enumerated in §5.2 (`zip-0226.rst`:287–291). Burn entry points reject duplicate Assets and amounts that do not fit in 63 bits (`add_burn`, orchard-zsa, `src/builder.rs`).

4.6 The Action statement, extended

A bin statement first. ZIP-226's "Circuit Statement" section describes the constraint changes informally and defers "all modifications in the Circuit" to an external document; no statement at the rigor of the *Ironwood Guide*'s Definition "Orchard Action statement, Ironwood circuit (NU6.3)" exists in any specification. The implemented circuit is the de facto specification, and every gate-level claim

below cites it (orchard-zsa, `src/circuit/circuit_zsa.rs`). Bin: *designed and implemented, but unspecified*.

ZIP-226 states the deltas against A1–A9 as follows:

(a) the Asset Base is witnessed once as an auxiliary input, and both note commitment integrity constraints (A1, A2) switch to $\mathsf{NoteCommit}^{\mathsf{OrchardZSA}}$ with that same witness — one variable simultaneously enforces input/output Asset equality and commitment correctness;

(b) an auxiliary boolean is_native_asset is constrained to equal 1 exactly when $\mathsf{AssetBase} = V^{\mathsf{Orchard}}$;

(c) $\mathsf{enableZSA} = 0$ forces $\mathsf{is\_native\_asset} = 1$;

(d) the value commitment's fixed-base multiplication (A4) becomes a variable-base multiplication by the witnessed Asset Base;

(e) a boolean split_flag replaces the spent value inside the value commitment: the committed net value becomes $v' - v_{\text{new}}$, with $v' = 0$ if $\mathsf{split\_flag} = 1$ and $v' = v_{\text{old}}$ otherwise; $\mathsf{split\_flag} = 1$ forces $\mathsf{is\_native\_asset} = 0$;

(f) the Merkle path constraint (A3) becomes $(v_{\text{old}} = 0 \wedge \mathsf{is\_native\_asset} = 1) \vee \text{ValidMerklePath}$ — the dummy skip survives only for the native Asset;

(g) nullifier integrity is randomized for split notes (§5).

The circuit realizes these deltas in a single gate, `q_orchard`, whose twelve named constraints are stated once, in §5.5, in the code's names (the code names the ZIP's is_native_asset as is_zatoshi_asset); we do not restate them here.

Two gadgets carry the arithmetic weight. The in-circuit value commitment computes $[\text{magnitude}]\,\mathsf{asset\_base}$ by variable-base long scalar multiplication on the witnessed non-identity point, applies mul_sign for the sign, and adds the fixed-base $[\mathsf{rcv}]\,R^{\mathsf{Orchard}}$ (`src/circuit/value_commit_orchard.rs`). The in-circuit note commitment shares one Sinsemilla evaluation between the two branches of Definition 4.2: it muxes the initial point Q between Q_{ZEC} (domain `z.cash:Orchard-NoteCommit-M`) and Q_{ZSA} (domain `z.cash:ZSA-NoteCommit-M`; the hard-coded point is test-asserted in `src/constants/sinsemilla.rs`) on is_zatoshi_asset, hashes the common message prefix once, hashes the two suffixes (ZEC: the zero-padded piece h_2; ZSA: the Asset Base bits), muxes the resulting hash point, and adds the shared blinding term $[\mathsf{rcm}]\,R$ — the shared randomness base of Definition 4.2 is what makes this sharing possible (`src/circuit/note_commit.rs`). The Asset Base message pieces: $h_{\text{ZSA}} = (\text{bits 249–253 of } \psi) \,\|\, (\text{bit 254 of } \psi) \,\|\, (\text{bits 0–3 of } x(\text{asset}))$; $i = \text{bits 4–253 of } x(\text{asset})$; $j = (\text{bit 254 of } x(\text{asset})) \,\|\, (\tilde{y}(\text{asset})) \,\|\, 8 \text{ zero bits}$ (ibid.).

The public instance grows from 9 to 10 field elements:

$$\big(\mathsf{anchor},\ \mathsf{cv}^{\mathsf{net}}.x,\ \mathsf{cv}^{\mathsf{net}}.y,\ \mathsf{nf}_{\text{old}},\ \mathsf{rk}.x,\ \mathsf{rk}.y,\ \mathsf{cmx},\ \mathsf{enable\_spends},\ \mathsf{enable\_outputs},\ \mathsf{enable\_zsa}\big),$$

with the new enable_zsa last. In the vanilla flavor enable_zsa is always false, and since halo2 pads instances with zeros, old nine-element proofs and new proofs behave identically; the crate accordingly carries two circuit flavors, `OrchardVanilla` and `OrchardZSA` (orchard-zsa, `src/circuit.rs`, `src/flavor.rs`). At the bundle level the enableZSAs flag is bit 2 of flagsOrchard (LSB order, after enableSpendsOrchard and enableOutputsOrchard; the implementation constant is `FLAG_ZSA_ENABLED = 0b0000_0100`), and coinbase transactions MUST set enableSpendsOrchard and enableZSAs to 0 (withdrawn ZIP-230; orchard-zsa, `src/bundle.rs`).

4.7 The carrier: transaction format, digests, and fees

The byte-level carrier has no merged deployment path upstream. Upstream ZIP-230 (the OrchardZSA v6 transaction format) is Status: Withdrawn, as is upstream ZIP-246 (its digests), and the new ZIP-229 (Version 6 Transaction Format; Draft, created 2026-06-13) states as a non-requirement that "the v6 transaction format need not support ZSAs". The QEDit fork retains ZIP-230 and ZIP-246 as Draft. We therefore document the carrier from the withdrawn upstream texts, the fork, and the implementation, and bin it accordingly: the transfer/burn cryptography above is Draft-specified, while the encoding it rides on is *designed and implemented but not specified upstream* (§1).

Note plaintext. The OrchardZSA note plaintext adds asset_base (32 bytes) after rseed. Per withdrawn ZIP-230 the v6 Orchard note plaintext is

$$\mathsf{np} := \left[\mathsf{leadByte}\right] \,||\, \mathrm{LEBS2OSP}_{88}(d) \,||\, \mathrm{I2LEOSP}_{64}(v) \,||\, \mathsf{rseed} \,||\, \mathsf{asset\_base} \,||\, K^{\mathsf{memo}},$$

with a 32-byte memo key K^{memo} replacing the 512-byte memo field, giving encCiphertext of 132 bytes and an OrchardZSAAction of 372 bytes (cv 32 + nullifier 32 + rk 32 + cmx 32 + ephemeralKey 32 + encCiphertext 132 + outCiphertext 80). ZIP-230 notes that an Orchard note "is essentially equivalent to an OrchardZSA note with AssetBase $= V^{\mathsf{Orchard}}$" and that the two need not be distinguished in plaintexts.

Remark 4.6 (The lead byte is unassigned upstream)**.** Upstream ZIP-226 requires leadByte to be the unresolved placeholder `{{ZSALEADBYTE}}` when the protocol's note-creation sections (§4.7.3 "Sending Notes", §4.8.3 "Dummy Notes") are invoked in the computation of ρ and ψ for OrchardZSA notes — a sentence with no counterpart in the fork's ZIP-226 and no implementation counterpart. Upstream ZIP-230 replaced its original `0x03` with the placeholder `{{LEADBYTE}}` because ZIP-2005 separately claims lead byte `0x03`. The QEDit fork ZIPs still say `0x03`, and the implementation hard-codes NOTE_VERSION_BYTE_V3 = `0x03` with MEMO_SIZE = 512 and no K^{memo} — the pre-memo-bundle layout, giving COMPACT_NOTE_SIZE_ZSA = 52 + 32 = 84 and a 612-byte encCiphertext, diverging from ZIP-230's 132 bytes (orchard-zsa, `src/primitives/zcash_note_encryption_domain.rs`, `src/primitives/orchard_primitives.rs`). No lead byte is currently reserved for ZSA notes upstream. Bin: *open*.

Digests. Withdrawn ZIP-246 extends the ZIP-244 digest tree — the baseline is the *Ironwood Guide*, "What the signatures sign: the ZIP-244 digest and its v6 form" — with a per-Action-Group orchard_burn_digest (node T.4a.vi) under orchard_action_groups_digest (personalization `ZTxIdOrcActGHash`), beside the compact and non-compact action digests, flagsOrchard, anchorOrchard, and nAGExpiryHeight. The compact digest covers nullifier, cmx, ephemeralKey, and encCiphertext (personalization `ZTxId6OActC_Hash`); the non-compact digest covers cv, rk, and outCiphertext (`ZTxId6OActN_Hash`). The burn digest is BLAKE2b-256 with personalization `ZTxIdOrcBurnHash` over, per assetBurn tuple,

$$\mathsf{assetBase} \,||\, \mathsf{valueBurn} \quad \text{(64-bit unsigned little-endian)},$$

the empty burn set hashing to BLAKE2b-256(`ZTxIdOrcBurnHash`, [ ]) (ZIP-246). One disambiguation: the header-digest field T.1g burn_amount is ZIP-233's network-sustainability burn, unrelated to asset burn (ZIP-246, T.1; ZIP-230, Transaction Format).

Parsing and bundle types. Function `read_v6_bundle` (gated by `zcash_unstable = "nu7"`) reads nActionGroups, which must be 0 or exactly 1 ("A V6 transaction must contain at most one action group"), then the actions, flags, anchor, nAGExpiryHeight (must be 0), the burn vector (32-byte Asset Base plus note value), the proof, per-action versioned signatures, valueBalance, and a versioned binding signature; the write path is symmetric via `write_burn` (librustzcash-zsa,

`zcash_primitives/src/transaction/components/orchard.rs`). The bundle type carries the burn set as the field `burn: Vec<(AssetBase, NoteValue)>` beside `value_balance`, and `binding_validating_key()` returns `derive_bvk(actions, value_balance, burn)`; the builder collects burns in a `BTreeMap<AssetBase, NoteValue>`, canonically ordered and duplicate-free by construction (orchard-zsa, `src/bundle.rs`, `src/builder.rs`). One open seam: ZIP-230 permits nActionGroupsOrchard > 1 with per-group burn fields (the multi-party rationale of §4.4), the implementation rejects more than one group, and ZIP-226's bvk equation is written over a single flat action/burn set; how multi-group aggregation would interact with the binding signature is unexercised. Bin: *open*.

Tree ordering. Per Action Group, every new OrchardZSA note commitment is appended to the note commitment tree per Action; afterwards, Issue Note commitments are appended per Issue Action (ZIP-227, "Addition to the Note Commitment Tree"; §3).

Fees. The QEDit fork of ZIP-317 adds contribution_ZSAIssuance and contribution_ZSACreation terms — issue notes and asset creations respectively — to the conventional-fee formula; it adds no burn term, so a burn contributes to fees only through the Actions that fund it. Upstream ZIP-317 contains no ZSA changes. The conventional-fee baseline is the *Wallet Guide*, "Fees (ZIP-317)".

5 Split notes and the counterfeiting boundary

An Orchard bundle pads to a uniform shape with *dummy notes*: an Action with no real input spends a zero-valued note under a freshly sampled key, and the membership constraint waves it through because the check is gated by the spent value — constraint A3 of the Action statement, $v_{\text{old}} \cdot (\text{root} - \text{anchor}) = 0$, "how an Action pads with no real input" (the *Ironwood Guide*, "The Action statement, formally"; its node-verification section, "What a node verifies, without ever observing the note", records that zero-value dummy spends are accepted by design). The construction is protocol §4.8.3, implemented in `orchard-zsa`, `src/builder.rs`, `SpendInfo::dummy`: a random spending key, a zero-valued note, and a random Merkle path that is never checked. The exemption is sound there because every Orchard value commitment rides the *fixed* base V^{Orchard} (*Ironwood Guide*, "Conservation without disclosure: value commitments and the binding signature"): a dummy spend can move no value, whatever path it fails to prove. ZIP-226 replaces the fixed base with a witnessed one, and the same exemption becomes a mint. This section develops the attack that forces the change, the *split note* that ZIP-226 substitutes for the dummy note on custom assets, and the exact constraint set — specified and implemented — that draws the counterfeiting boundary.

Ground truth for this section: ZIP-226 and ZIP-227 at `zcash/zips` commit `69610984` (2026-07-09); the pinned implementation, crate `orchard` of the QED-it fork (`orchard-zsa`, branch `zsa1`) at commit `cf801a5d` (2026-06-29); the QED-it `zips` fork at `fd71419a` where it diverges from upstream. Split-note logic lives entirely in the `orchard` crate; the `librustzcash` fork consumes its bundle API and contains no split-note code of its own (verified by search at `3f9b53fc`).

5.1 The counterfeit construction the boundary excludes

ZIP-226 generalises the net value commitment from the fixed base V^{Orchard} to a per-asset base (ZIP-226, "Value Commitment"):

$$\begin{aligned} \mathsf{cv}^{\mathsf{net}} &:= \mathsf{ValueCommit}_{\mathsf{rcv}}^{\mathsf{OrchardZSA}}\big(\mathsf{AssetBase}_{\mathsf{AssetId}},\, v^{\mathsf{net}}_{\mathsf{AssetId}}\big) \\ &= [\, v^{\mathsf{net}}_{\mathsf{AssetId}} \,]\, \mathsf{AssetBase}_{\mathsf{AssetId}} + [\mathsf{rcv}]\, R^{\mathsf{Orchard}}, \end{aligned} \tag{1}$$

with $v^{\mathsf{net}} := v_{\text{old}} - v_{\text{new}}$ and R^{Orchard} the randomness base of the *Ironwood Guide*'s net value commitment ("Conservation without disclosure: value commitments and the binding signature"),

unchanged. The native asset keeps the old base: "the Asset Base of the ZEC Asset will be kept as the original value base point $\mathcal{V}^{\mathsf{Orchard}}$" (ZIP-226, "Asset Identifiers"; in code `AssetBase::zatoshi()` is $\mathsf{GroupHash}^{\mathsf{Pallas}}$(`z.cash : Orchard-cv, "v"`), `orchard-zsa`, `src/note/asset_base.rs`, `src/constants/fixed_bases.rs`). Verification subtracts the declared native balance and the declared burns (ZIP-226, "Value Balance Verification"; implemented in `src/bundle.rs`, `derive_bvk_raw`):

$$\begin{aligned}\mathsf{bvk} = \Big(\sum_{i=1}^{n} \mathsf{cv}_i^{\mathsf{net}}\Big) - \mathsf{ValueCommit}_0^{\mathsf{OrchardZSA}}\big(V^{\mathsf{Orchard}},\ v^{\mathsf{balanceOrchard}}\big) \\ - \sum_{(\mathsf{AssetBase},\, v) \in \mathsf{assetBurn}} \mathsf{ValueCommit}_0^{\mathsf{OrchardZSA}}(\mathsf{AssetBase},\ v),\end{aligned} \tag{2}$$

and the binding signature under bvk proves knowledge of $\mathsf{bsk} = \sum_i \mathsf{rcv}_i$ exactly as in Orchard.

Suppose now that the dummy exemption survived unchanged: a padding Action proves no membership, and — since the base of (1) is a private witness — may witness *any* point A as its asset base. Choose $A := [-1]\,V^{\mathsf{Orchard}}$, witness $v_{\mathrm{old}} = 0$ and $v_{\mathrm{new}} = v$:

$$\mathsf{cv}_1^{\mathsf{net}} = [\,0 - v\,]\,A + [\mathsf{rcv}_1]\,R^{\mathsf{Orchard}} = [\,v\,]\,V^{\mathsf{Orchard}} + [\mathsf{rcv}_1]\,R^{\mathsf{Orchard}}.$$

Pair it with a second Action that outputs a genuine v-valued ZEC note,

$$\mathsf{cv}_2^{\mathsf{net}} = [\,0 - v\,]\,V^{\mathsf{Orchard}} + [\mathsf{rcv}_2]\,R^{\mathsf{Orchard}},$$

and declare $v^{\mathsf{balanceOrchard}} = 0$ with no burns. The V^{Orchard}-components cancel in (2),

$$\mathsf{bvk} = \mathsf{cv}_1^{\mathsf{net}} + \mathsf{cv}_2^{\mathsf{net}} = [\mathsf{rcv}_1 + \mathsf{rcv}_2]\,R^{\mathsf{Orchard}},$$

the attacker knows $\mathsf{bsk} = \mathsf{rcv}_1 + \mathsf{rcv}_2$, the binding signature verifies — and a v-valued ZEC note now exists that no one paid for. Any known multiple or linear combination of existing bases serves equally. (*Classification:* the construction is assembled here from the specified formulas (1) and (2); ZIP-226 states the attack only in words.) The ZIP's own statement, verbatim: the output note of split Actions "cannot contain just any Asset Base. We must enforce it to be an actual output of a GroupHash computation... Without this enforcement the prover could input a multiple (or linear combination) of an existing Asset Base, and thereby attack the network by overflowing the ZEC value balance and hence counterfeiting ZEC funds" (ZIP-226, "Split Notes", rationale).

The boundary to draw is therefore precise: every asset base that enters a value commitment must be proven to be an output of GroupHash — a point of unknown discrete-log relation to V^{Orchard} and R^{Orchard} — and the one place an unproven base could enter is an Action with no real input.

5.2 The split input

> **Definition 5.1** (Split Input, Split Action)**.** ZIP-226 defines a *Split Input* as "an Action input used to ensure that the output note of that Action is of a validly issued AssetBase when there is no corresponding real input note, in situations where the number of outputs are larger than the number of inputs", and a *Split Action* as an Action containing a Split Input (ZIP-226, "Terminology").

Mechanically, a Split Input is a *copy of a previously issued note* — one whose commitment is already in the note commitment tree — with exactly two changes: a boolean witness split_flag is set to 1, and the note's value is replaced by 0 *in the value-commitment computation only* (ZIP-226, "Split Notes"). The copied note's commitment $\mathsf{cm}_{\mathrm{old}}$ still opens with the true value: in the circuit the value is zeroed solely in the $\mathsf{cv}^{\mathsf{net}}$ equation, while the old-note commitment check consumes v_{old} unchanged (`orchard-zsa`, `src/circuit/circuit_zsa.rs`: the gate zeroes v_{old} via the $(1 - \mathsf{split\_flag})$ factor in the value equation, lines 127–131, whereas the NoteCommit call at lines 637–655 takes v_{old} as witnessed). The copy is therefore not a spend — it moves no value and, as §5.6 shows, reveals no usable

nullifier — but it drags the original note's asset base into the proof, where an equality constraint passes it to the output note.

The split Action still balances, per asset. Committed values must sum to zero for each asset base separately, "as the Pedersen commitments add up homomorphically only with respect to the same value base point" (ZIP-226, "Value Balance Verification"): for a correct transaction the custom-asset commitments satisfy

$$\sum_{j \in S_{\mathsf{CA}}} \mathsf{cv}_j^{\mathsf{net}} - \sum_{(\mathsf{AssetBase},\, v) \,\in\, \mathsf{assetBurn}} [\, v \,]\, \mathsf{AssetBase} = \sum_{j \in S_{\mathsf{CA}}} [\mathsf{rcv}_j]\, R^{\mathsf{Orchard}},$$

so the split Action's $\mathsf{cv}^{\mathsf{net}} = [\, 0 - v_{\mathrm{new}} \,]\, \mathsf{AssetBase} + [\mathsf{rcv}]\, R^{\mathsf{Orchard}}$ cancels against the real spending Actions of the same asset. The builder implements exactly this: `ActionInfo::value_sum` uses a spent value of 0 when split_flag is set (`orchard-zsa`, `src/builder.rs`, lines 495–506), and `derive_bvk_raw` computes (2) over the resulting commitments (`src/bundle.rs`, lines 482–517).

For the note to copy, ZIP-226 offers the wallet three options: (1) another note of the same Asset Base being spent in the same transaction; (2) a different unspent note of that Asset Base, which is *not* thereby spent; (3) an already spent note of that Asset Base — the zeroed value prevents double spending (ZIP-226, "Split Notes"). The ZIP adds that "we do not care about whether the previously issued note copied to create a Split Input is owned by the sender, or whether it was nullified before"; §5.7 examines how far that sentence actually holds against the constraint system.

5.3 Membership as well-formedness: the issuance anchor

The fix ZIP-226 specifies, verbatim: "for Custom Assets we enforce that every input note to an OrchardZSA Action must be proven to exist in the set of note commitments in the note commitment tree. We then enforce this real note to be unspendable in the sense that its value will be zeroed in split Actions and the nullifier will be randomized... the proof itself ensures that the output note is of the same Asset Base as the input note. In the circuit, the split note functionality will be activated by a boolean private input (aka the split_flag boolean)" (ZIP-226, "Split Notes", rationale).

Why does tree membership prove that an asset base is a GroupHash output? Because for custom assets the tree is fed exclusively from two sources that both enforce it. Transfer outputs inherit their base from a proven input by the in-circuit equality constraint (§5.4). Issuance outputs are checked publicly: a consensus rule obliges validators to recompute each Issue Note's AssetBase from the issuance bundle's issuer key and the IssueAction's assetDescHash (ZIP-227, "Specification: Consensus Rule Changes"), along the derivation chain constructed in §2.3 (ZIP-227, "Asset Bases"; recomputed in `orchard-zsa`, `src/note/asset_base.rs`), and then appends the issue-note commitments to the *same* Orchard note commitment tree — transfer-Action commitments first, then issuance notes (ZIP-227, "Addition to the Note Commitment Tree"). Hence a custom-asset note commitment resident in the tree commits, by induction along the two rules, to a genuine GroupHash output. ZIP-226 states the conclusion: "This ensures that the value base points of all output notes of a transfer are actual outputs of a GroupHash, as they originate in the Issuance protocol which is publicly verified" ("Split Notes", rationale); and, on the identifier side, "We prevent a potential malleability attack on the Asset Identifier by ensuring the output notes receive an Asset Base that exists on the global state" ("Rationale for Asset Identifiers").

One clarification the ZIP's sentence invites: "every input note to an OrchardZSA Action" is scoped to *custom-asset* inputs. Native-asset (ZEC) Actions in a v6 bundle retain the dummy-note exemption unchanged — "The ZEC-based Actions will still include dummy input notes, whereas the OrchardZSA Actions will include split input notes and will not include dummy input notes" (ZIP-226, "Backward Compatibility"; the "Split Notes" rationale states the same). The constraint-level form of this scoping is the disjunction in §5.4.

5.4 The specified circuit delta

We state ZIP-226's circuit changes ("Circuit Statement") as a delta against the Action statement A1–A9 of the *Ironwood Guide* ("The Action statement, formally"). Two new boolean witnesses appear: split_flag, and is_native_asset, constrained to equal 1 exactly when the witnessed base is the native one, $\mathsf{AssetBase} = V^{\mathsf{Orchard}}$. One public input appears: the bundle-level enableZSA flag (a bit of the v6 flagsOrchardZSA field), with the constraint $\mathsf{enableZSA} = 0 \implies \mathsf{is\_native\_asset} = 1$ — switching the flag off collapses the circuit to native-asset behaviour and, transitively via the split constraint below, disables split notes (ZIP-226, "Overview" and "Circuit Statement", The enableZSA Flag; in code the flag is the tenth public input, taken from `Flags::zsa_enabled()`, `orchard-zsa`, `src/circuit.rs`, lines 476–494, 560).

- **A1, A2 (note commitment integrity).** Both commitment checks replace $\mathsf{NoteCommit}^{\mathsf{Orchard}}$ with $\mathsf{NoteCommit}^{\mathsf{OrchardZSA}}$, which takes the witnessed AssetBase as an additional final argument; the *same* once-witnessed base feeds the old-note check, the new-note check, and the value commitment (ZIP-226, "Circuit Statement", Asset Base Equality). For the native asset $\mathsf{NoteCommit}^{\mathsf{OrchardZSA}}$ coincides with $\mathsf{NoteCommit}^{\mathsf{Orchard}}$; otherwise it is the Sinsemilla commitment over the extended message ending in the asset-base encoding, in the hash domain `"z.cash:ZSA-NoteCommit-M"` with the blinding base shared with Orchard (`"z.cash:Orchard-NoteCommit-r"`) (ZIP-226, "Note Structure and Commitment"). The full $\mathsf{NoteCommit}^{\mathsf{OrchardZSA}}$ construction is Definition 4.2; its in-circuit mux is described in §4.6.

- **A3 (membership).** The value gate becomes a disjunction (ZIP-226, "Circuit Statement", Asset Identifier Consistency for Split Actions — Merkle Path Validity):

$$\big(v_{\text{old}} = 0 \ \wedge\ \mathsf{is\_native\_asset} = 1\big) \ \vee\ \big(\text{Valid Merkle Path}\big),$$

  so the dummy skip survives *only* for the native asset. Every custom-asset input — split or not, zero-valued or not — must prove membership.

- **A4 (value commitment).** Two changes. First, "the fixed-base multiplication constraint between the value and the value base point of the value commitment, cv, is replaced with a variable-base multiplication between the two" (ZIP-226, "Circuit Statement", Value Commitment Correctness), the base being the witnessed AssetBase. Second, the input value is replaced by a generic v' (ZIP-226, "Circuit Statement", Asset Identifier Consistency for Split Actions):

$$\mathsf{cv}^{\mathsf{net}} = \mathsf{ValueCommit}_{\mathsf{rcv}}^{\mathsf{OrchardZSA}}\big(\mathsf{AssetBase},\ v' - v_{\text{new}}\big), \qquad v' = \begin{cases} 0 & \text{if split\_flag} = 1, \\ v_{\text{old}} & \text{otherwise,} \end{cases}$$

  together with $\mathsf{split\_flag} = 1 \implies \mathsf{is\_native\_asset} = 0$: split notes exist only for custom assets, and a zero-valued native dummy cannot masquerade as one.

- **A5 (nullifier).** The derivation changes for split notes "to prevent the identification of notes" (ZIP-226, "Circuit Statement", Asset Identifier Consistency for Split Actions); §5.6 develops it.

- **A6, A7 (spend authority, address integrity).** Untouched by the ZIP's change list — and, decisively for §5.7, they remain *unconditional*: no split_flag factor relaxes them.

5.5 The implemented gate

A naming note, once: the ZIP's is_native_asset is `is_zatoshi_asset` throughout the implementation, assigned 1 exactly when the witnessed asset equals `AssetBase::zatoshi()` (`orchard-zsa`, `src/circuit/gadget.rs`, `circuit_zsa.rs`). We keep the ZIP name in prose and the code name inside code-derived formulas.

The implemented circuit concentrates the delta in a single gate, `q_orchard`, which carries twelve named constraints where the vanilla circuit's corresponding gate has four (`src/circuit/circuit_zsa.rs`, lines 68–185, versus `src/circuit/circuit_vanilla.rs`, lines 59–100). In the code's names:

$$\mathsf{bool\_check}(\mathsf{split\_flag}), \qquad \mathsf{bool\_check}(\mathsf{is\_zatoshi\_asset}),$$
$$v_{\text{old}} \cdot (1 - \mathsf{split\_flag}) - v_{\text{new}} = \mathsf{magnitude} \cdot \mathsf{sign},$$
$$(v_{\text{old}} + 1 - \mathsf{is\_zatoshi\_asset}) \cdot (\mathsf{root} - \mathsf{anchor}) = 0,$$
$$v_{\text{old}} = 0 \vee \mathsf{enable\_spends} = 1, \qquad v_{\text{new}} = 0 \vee \mathsf{enable\_outputs} = 1,$$
$$\mathsf{is\_zatoshi\_asset} \cdot (\mathsf{asset}_x - \mathsf{zatoshi}_x) = 0, \qquad \mathsf{is\_zatoshi\_asset} \cdot (\mathsf{asset}_y - \mathsf{zatoshi}_y) = 0,$$
$$(1 - \mathsf{is\_zatoshi\_asset}) \cdot (\Delta_x \cdot \Delta_x^{\mathsf{inv}} - 1) \cdot (\Delta_y \cdot \Delta_y^{\mathsf{inv}} - 1) = 0,$$
$$(1 - \mathsf{split\_flag}) \cdot (\psi_{\text{old}} - \psi^{\mathsf{nf}}) = 0, \qquad \mathsf{split\_flag} \cdot \mathsf{is\_zatoshi\_asset} = 0,$$
$$(1 - \mathsf{enable\_zsa}) \cdot (1 - \mathsf{is\_zatoshi\_asset}) = 0,$$

where Δ_x, Δ_y abbreviate the code's `diff_asset_x`, `diff_asset_y` — the witnessed asset's coordinates minus the native base's — and ψ^{nf} is the code's `psi_nf` (§5.6). Four points merit unpacking.

The folded value equation. The single constraint $v_{\text{old}} \cdot (1 - \mathsf{split\_flag}) - v_{\text{new}} = \mathsf{magnitude} \cdot \mathsf{sign}$ folds the ZIP's case-split v' into one expression; the witness generator supplies the matching net value, $-v_{\text{new}}$ for split spends and $v_{\text{old}} - v_{\text{new}}$ otherwise (`circuit_zsa.rs`, lines 462–501).

The Merkle factor. The disjunction of A3 is encoded as the single product $(v_{\text{old}} + 1 - \mathsf{is\_zatoshi\_asset}) \cdot (\mathsf{root} - \mathsf{anchor}) = 0$. The code comment argues the encoding safe from wrap-around: $\mathsf{is\_zatoshi\_asset}$ is boolean-checked and v_{old} is range-checked to 64 bits in the note-commitment evaluation, so over $\mathbb{F}_{p_{\mathsf{Pallas}}}$ the first factor vanishes exactly when $v_{\text{old}} = 0$ and $\mathsf{is\_zatoshi\_asset} = 1$ (`circuit_zsa.rs`, lines 132–139). The consequence bears repeating: for every custom-asset input the factor is nonzero, so $\mathsf{root} = \mathsf{anchor}$ is mandatory — a strictly stronger membership requirement than vanilla Orchard's v_{old}-gate, applying even to zero-valued non-split custom-asset inputs.

Proving inequality. The assignment $\mathsf{is\_zatoshi\_asset} = 0$ must itself be sound: the circuit must verify that the witnessed base *differs* from V^{Orchard}. The prover witnesses field inverses $\Delta_x^{\mathsf{inv}}, \Delta_y^{\mathsf{inv}}$ (assigned 0 when the corresponding difference vanishes), and the triple product above forces at least one difference to be exhibited invertible (`circuit_zsa.rs`, lines 160–170 for the gate, 792–841 for the assignment).

The variable-base value commitment. With the base witnessed, the vanilla circuit's fixed-base *short* multiplication $[\,v^{\mathsf{net}}\,]\,V^{\mathsf{Orchard}}$ — whose 64-bit range check is implicit in the short-multiplication decomposition — is no longer available. The ZSA gadget makes the range check explicit: the magnitude is decomposed against the Sinsemilla lookup table as six 10-bit windows plus one 4-bit short check (64 bits), then promoted to a full-width scalar (`ScalarVar::from_base`) and fed to the variable-base long multiplication — the code notes it is "optimized for ~255-bit scalar" and leaves a TODO to specialise it for 64 bits — after which the sign is applied and the blind $[\mathsf{rcv}]\,R^{\mathsf{Orchard}}$ added (`src/circuit/value_commit_orchard.rs`, lines 38–127). Two supporting details: the ZSA circuit uses the extended lookup configuration `PallasLookupRangeCheck4_5BConfig` (an additional lookup-table column for 4- and 5-bit short checks), and it refuses to synthesise under `CircuitVersion::InsecureUnanchoredBase`, the pre-NU6.2 `halo2_gadgets` variable-base scalar-multiplication version whose base point was not anchored — directly load-bearing here, because the entire counterfeiting boundary rests on the multiplication being by the *witnessed* AssetBase and nothing else (`circuit_zsa.rs`, lines 22, 51, 190–197, 334–337; `circuit.rs`, lines 151–185).

The flag interactions are tested asymmetrically: the three legal combinations of (is_zatoshi_asset, split_flag) — $(1,0)$, $(0,1)$, and $(0,0)$ — are proven passing, while the $(1,1)$ combination is excluded at witness-generation time by the test helper's assertion ("We cannot create a split note with a zatoshi asset"), so the constraint $\mathsf{split\_flag} \cdot \mathsf{is\_zatoshi\_asset} = 0$ is not itself exercised by a failing-proof test (`circuit_zsa.rs`, lines 1174–1312). Documented cost: the ZSA proof measures 5120 bytes for one Action and 7392 for two, against the vanilla 4992 and 7264, at the shared circuit size $K = 11$; the enableZSA bit is the tenth public input, at index 9 (the proof-size tests of `circuit_zsa.rs` and `circuit_vanilla.rs`; `circuit.rs`, lines 68–80, 536–563).

5.6 The split nullifier

A split input copies a note that may sit unspent in someone's wallet; its Action must nevertheless publish *some* nullifier. Publishing the note's true nullifier would be harmful in both directions: it would mark an unspent note spent (bricking it), and it would collide with the honest nullifier if the same note were spent for real in the same transaction. ZIP-226 therefore randomises the split note's nullifier ("Split Notes"):

$$\mathsf{nf}_{\mathrm{old}} = \mathsf{Extract}\big([\, F_{\mathsf{nk}}(\rho_{\mathrm{old}}) + \psi^{\mathsf{nf}} \,]\, G^{\mathsf{nf}} \;+\; \mathsf{cm}_{\mathrm{old}} \;+\; L^{\mathsf{Orchard}} \big), \tag{3}$$

in the notation of the *Ironwood Guide*'s A5 ("The Action statement, formally"; ZIP-226 writes $\mathcal{K}^{\mathsf{Orchard}}$ for G^{nf} and computes the bracketed sum in the Pallas base field). Two deltas against A5: the note's ψ_{old} is replaced by a fresh ψ^{nf}, and a fixed offset point

$$L^{\mathsf{Orchard}} := \mathsf{GroupHash}^{\mathsf{Pallas}}(\texttt{z.cash : Orchard}, \texttt{"L"})$$

is added (ZIP-226, "Split Notes"; the constant is pinned and test-asserted against the GroupHash evaluation in `orchard-zsa`, `src/constants/nullifier_l.rs`). The gate constraint $(1-\mathsf{split\_flag})\cdot(\psi_{\mathrm{old}} - \psi^{\mathsf{nf}}) = 0$ of §5.5 makes the substitution split-only: non-split OrchardZSA nullifiers coincide with the Orchard derivation, as the ZIP states ("Note Structure and Commitment": the non-split nullifier "is generated in the same manner as in the Orchard protocol").

In circuit, the nullifier gadget computes the standard point $\mathsf{cm}_{\mathrm{old}} + [\, F_{\mathsf{nk}}(\rho_{\mathrm{old}}) + \psi^{\mathsf{nf}} \,]\, G^{\mathsf{nf}}$ from the witnessed ψ^{nf}, forms the offset variant by adding the constant L^{Orchard}, and muxes the two on split_flag before extraction (`src/circuit/derive_nullifier.rs`, lines 54–111). Out of circuit, `Nullifier::derive` mirrors the computation and selects the offset variant in constant time exactly when the note carries a `rseed_split_note` (`src/note/nullifier.rs`, lines 73–90; `src/note.rs`, lines 415–426).

Deriving ψ^{nf}: a three-way divergence. The value of ψ^{nf} is a private witness, constrained by the circuit only in the *non*-split case; how a wallet derives it for split notes is therefore wallet-level, not consensus-level. The three extant texts disagree; per the volume's policy (§1.3), the upstream ZIP formula comes first. Upstream ZIP-226 — written against the ZIP-2005 baseline, "expected to deploy before any ZSA activation" — specifies

$$\psi^{\mathsf{nf}} := \mathrm{ToBase}\big(\mathsf{PRFexpand}_{\mathsf{rseed\_nf}}(\texttt{0x0A} \,\|\, \rho_{\mathrm{old}})\big)$$

with rseed_nf uniform on 32 bytes and the domain byte `0x0A` "reserved by ZIP 2005 for this use" (`zip-0226.rst`:297 @ 69610984). The QED-it `zips` fork specifies neither derivation: there ψ^{nf} "is sampled uniformly at random" on the Pallas base field (fork `zip-0226.rst`:293 @ `fd71419a`). The implementation derives

$$\psi^{\mathsf{nf}} := \mathrm{ToBase}\big(\mathsf{PRFexpand}_{\mathsf{rseed\_split\_note}}(\texttt{0x09} \,\|\, \rho_{\mathrm{old}})\big),$$

the ordinary ψ domain byte keyed by a fresh random 32-byte `rseed_split_note` (`orchard-zsa`, `src/circuit.rs`:316–319, `src/note.rs`:114–116 and 417–425; domain byte $\mathtt{PSI} = \mathtt{0x09}$ per the QED-it `zcash_spec` fork, `src/prf_expand.rs`:89 @ `d5e84264`). All three yield a uniformly distributed witness; the implementation matches neither ZIP text exactly. (*Classification:* each derivation is specified in its own text; the divergence is consensus-invisible.)

Why the offset L^{Orchard}? The randomised ψ^{nf} already makes the split nullifier unlinkable to the note's true nullifier. ZIP-226 states no rationale for additionally adding L^{Orchard}. A plausible purpose is robustness against a prover who *chooses* $\psi^{\mathsf{nf}} = \psi_{\mathsf{old}}$ — the circuit does not forbid it in the split case — and would thereby publish the note's true nullifier from a zero-valued split Action, bricking the original note or colliding with an honest spend of it elsewhere in the transaction; the constant offset makes even that choice yield a different group element. (*Classification:* designed-but-unspecified; the preceding sentence is our inference, stated in neither ZIP nor code.)

5.7 Ownership, in fact and in prose

Recall the ZIP's claim that "we do not care about whether the previously issued note copied to create a Split Input is owned by the sender" (§5.2). The constraint system says otherwise. Spend authority (A6) and address integrity (A7) are unchanged *and unconditional* in the ZSA circuit: the proof establishes $\mathsf{rk} = [\alpha]\, G^{\mathsf{SpendAuth}} + \mathsf{ak}$ and $\mathsf{ivk} = \mathsf{Commit}^{\mathsf{ivk}}_{\mathsf{r_{ivk}}}(\mathsf{Extract}(\mathsf{ak}), \mathsf{nk})$ with $\mathsf{pk_{d_{old}}} = [\mathsf{ivk}]\, \mathsf{g_{d_{old}}}$ — for split inputs too (`orchard-zsa`, `src/circuit/circuit_zsa.rs`, lines 554–623; the old-note commitment equality at lines 625–659). Because the copied note's $\mathsf{cm_{old}}$ must open to the actual tree leaf (Sinsemilla binding plus the mandatory Merkle check of §5.5), the witnessed $\mathsf{pk_{d_{old}}}$ is the original recipient's, and A7 then forces the prover to know $(\mathsf{ak}, \mathsf{nk}, \mathsf{r_{ivk}})$ consistent with it — that is, to hold the copied note's full viewing key. The ZIP's ownership sentence holds for *spentness* — a nullified note is a perfectly good split input, precisely because the value is zeroed — but not for ownership in the key-material sense. The implementation never exercises the freedom the prose suggests: the builder only ever copies one of the sender's own spends in the same bundle, option (1) of §5.2 (`src/builder.rs`, `pad_spend`). (*Classification:* the constraint reading is specified — the ZIP's constraint list and the implemented circuit agree; the rationale prose overstates.)

One genuine exception exists, and neither ZIP states it. ZIP-227 requires the first note of any asset's first issuance to be a *reference note*: value 0, recipient the default diversified address of the all-zero Orchard spending key — a key anyone can derive (ZIP-227, "Reference Notes"; `orchard-zsa`, `src/constants/reference_keys.rs` pins the all-zero spending key and the 43-byte raw address). Because that spending key is public, anyone can satisfy A6/A7 for the reference note, which makes it a universally available split input for its asset — the one case in which "ownership does not matter" holds literally. (*Classification:* inference; the connection between reference notes and split inputs is stated in neither ZIP nor the implementation.)

5.8 Builder mechanics: padding, per asset

The two padding regimes coexist per ZIP-226's rule quoted in §5.3: dummy inputs for ZEC Actions, split inputs for custom-asset Actions. The builder partitions spends and outputs by asset, pads each side of each partition to equal length, shuffles within each asset, then shuffles the assembled Actions across assets (`orchard-zsa`, `src/builder.rs`, lines 1081–1164). The padding spend is asset-dependent (`pad_spend`, lines 928–944): for the native asset, `SpendInfo::dummy` exactly as in Orchard; for a custom asset, `create_split_spend` on the first real spend of that asset in the bundle — and if the bundle spends none, `BuildError::NoSplitNoteAvailable` ("No split note has been provided for this asset"): unlike a dummy, a split input cannot be conjured from nothing. Output-side padding uses dummy outputs of the padding asset in both regimes.

The copy itself is minimal. Function `Note::create_split_note` duplicates the note and sets exactly one new field, rseed_split_note — a fresh random 32-byte seed, drawn by `RandomSeed::random` and resampled until the derived esk is valid — asserting the asset is not the native one; `SpendInfo::create_split_spend` keeps the original note's full viewing key and Merkle path and raises split_flag (`src/note.rs`, lines 436–442, 95–103; `src/builder.rs`, lines 302–318). Recipient, value, asset, ρ, rseed — hence $\mathsf{cm_{old}}$ — are identical to the tree-resident original; the only witness-level differences downstream are split_flag $= 1$ and the ψ^{nf} drawn from the new seed. Witness assembly routes exactly that: `Witnesses::from_action_context_unchecked` takes

ψ^{nf} from rseed_split_note when present and from rseed otherwise — so $\psi^{\mathsf{nf}} = \psi_{\mathrm{old}}$ for every non-split spend — and packs (ψ^{nf}, asset, split_flag) into the ZSA witness extension that the vanilla flavour leaves unknown (`src/circuit.rs`, lines 199–244, 300–345).

The counterfeiting boundary, restated once in full: a custom-asset output note's base equals its Action's input-note base (the A1/A2 equality on the shared witness); that input note is a tree leaf (the mandatory Merkle factor); every custom-asset tree leaf descends from a publicly recomputed GroupHash output (the issuance anchor of §5.3); and the copy that makes padding possible moves no value (the zeroed v') and emits an unlinkable, non-colliding nullifier (3). Each link is enforced by a constraint quoted above; breaking any one of them re-opens the mint of §5.1.

6 Transaction integration and outlook

The preceding sections construct the OrchardZSA objects — asset bases (§2), notes and Actions (§4, §5), issuance (§3), and burn (§4.4). This section places them in a transaction: the v6 wire format (ZIP-230), the digest tree that yields transaction identifiers, sighashes, and the authorizing-data commitment (ZIP-246), the transaction-level consensus rules of ZIP-226/227, the fee treatment (ZIP-317), the state of the QED-it implementation stack at commit-pinned sources, the obligations the format imposes on wallets, and the deployment outlook. Deployment status — Draft ZIPs, externally audited at circuit scope, running on QED-it's public testnet, contested for NU7 — is stated once in the scope section (§1.2) and not relitigated per object; what this section adds is the classification of each format artifact, because the ZSA transaction layer is the part of the protocol where specification and implementation have diverged furthest. Table 4 pins the sources; all repository facts below were verified firsthand at the stated commits, on 2026-07-14.

Source	Commit	Date	Role
`zcash/zips`	`6961098`	2026-07-09	upstream ZIP texts
QED-it `zips` fork	`fd71419`	2026-02-11	fork ZIP texts (pre-withdrawal)
QED-it `orchard` fork, `zsa1`	`cf801a5d`	2026-06-29	OrchardZSA bundle, circuit, issuance
QED-it `librustzcash` fork, `zsa1`	`3f9b53fc`	2026-04-17	v6 assembly, digests, fees
`zcash/orchard` @ `ltf/zsa`	`a799082e`	2025-11-25	upstream tracking branch

Table 4: Pinned sources for this section. The `ltf/zsa` branch merges ZSA-compatible Halo 2 plus PCZT external spend-authorization signatures; it is the beginning of an upstreaming path, not a deployable stack.

6.1 Format status: version 6 no longer means OrchardZSA

The carrier format for everything in this volume is ZIP-230, the OrchardZSA v6 transaction format, with its digests in ZIP-246. Both are *withdrawn* upstream. ZIP-230 is retitled "Withdrawn Version 6 Transaction Format"; its warning states that it is obsoleted by ZIP-229 and "will not be deployed. Transaction version number 6 is now defined by ZIP 229." (`zip-0230.rst`, header and warning). ZIP-246 is likewise withdrawn ("Digests for the Withdrawn Version 6 Transaction Format"); v6 transaction identifiers, authorizing-data commitments, and signature digests are now specified by ZIP-229 (`zip-0246.rst`, header). The QED-it fork copies of both remain Status: Draft under their original titles — the fork (@ `fd71419`) predates the withdrawal.

ZIP-229 (Status: Draft, created 2026-06-13) redefines v6 for NU6.3, introducing the Ironwood pool in the aftermath of the Orchard soundness vulnerability whose mitigation deployed as NU6.2 (ZIP-257, Status: Final). Its Non-requirements state explicitly that "the v6 transaction format need not support ZSAs, the `zip233Amount` field, the explicit fee field, or per-transparent-input sighash information that appeared in the withdrawn ZIP 230," and that version number 6 need not avoid collision

with withdrawn ZIP-230 or draft ZIP-248 (ZIP-229, Non-requirements). The same transaction version number therefore now denotes two incompatible byte formats, and the ZSA one is the orphan.

Classification, per the three-bin discipline of this series (the *Tachyon Guide*, §"The three-bin method"): the byte-level format below is fixed at ZIP rigor only by withdrawn upstream text and pinned fork code — *designed but unspecified* (§1.4) — with no deployment path under its current ZIP numbers. We document it because it is the format the pinned implementation speaks and the reference against which any respecification will be diffed; §6.8 states what would have to happen for it to become normative again.

6.2 The v6 wire format (ZIP-230)

Common and transparent fields. The common header carries `header` (`fOverwintered` and the version), `nVersionGroupId`, `nConsensusBranchId`, `lock_time`, `nExpiryHeight`, an explicit 8-byte `fee` (`uint64`, per ZIP-2002), and an 8-byte `zip233Amount` (`uint64`, per ZIP-233) (ZIP-230, Transaction Format). Making the fee an explicit field — rather than the implicit difference between transparent inputs and outputs — is one of the format's two departures from v5 visible already in the header; the second is that signatures become versioned, below. The transparent fields add `vSighashInfo`: one `TransparentSighashInfo` (a `compactSize` length followed by `sighashInfo` bytes) per transparent input (ZIP-230, Transaction Format).

Orchard fields: Action Groups. The Orchard component becomes a vector of *Action Groups*: `nActionGroupsOrchard`, `vActionGroupsOrchard`, then a transaction-level `valueBalanceOrchard` (`int64`) and `bindingSigOrchard`. The balance and binding signature are present iff `nActionGroupsOrchard` > 0; an absent `valueBalanceOrchard` means $v^{\text{Orchard}}_{\text{balance}} = 0$ (ZIP-230, Transaction Format). Each `ActionGroupDescription` contains, in order (ZIP-230, OrchardZSA Action Group Description):

- `nActionsOrchard` (`compactSize`, MUST be > 0) and `vActionsOrchard`, at 372 bytes per `OrchardZSAAction`;
- `flagsOrchard`, LSB-first: `enableSpendsOrchard`, `enableOutputsOrchard`, `enableZSAs`, remaining bits zero;
- `anchorOrchard` (32 bytes) — each group carries its own anchor;
- `nAGExpiryHeight` (`uint32`);
- `nAssetBurn` and `vAssetBurn`, at 40 bytes per `AssetBurn` entry;
- `sizeProofsOrchard` and `proofsOrchard`, one aggregated proof for the group;
- `vSpendAuthSigsOrchard`, one `OrchardSignature` per Action.

Field `nAGExpiryHeight` exists for forward compatibility with ZSA atomic swaps. ZIP-228 is now a Draft that uses this field to expire swap orders, but it still depends on the withdrawn ZIP-230/246 carrier; for the core OrchardZSA proposal the field is zero by consensus (ZIP-230, Rationale for `nAGExpiryHeight`; §6.4). A normative rule of ZIP-230 (its consensus-rule list, not the rationale) states "In NU7, `nExpiryHeight` MUST be set to 0" — read literally, a constraint on the transaction-level ZIP-203 field that would contradict ordinary expiry semantics; context, the ZIP-227 consensus rule, and the implementation (which constrains only `nAGExpiryHeight`) all indicate `nAGExpiryHeight` is meant. We flag this as an apparent editorial defect in the withdrawn text. Burn fields sit inside each Action Group rather than at transaction level so that, in a future multi-party Action Group world, each party can burn assets within its own group (ZIP-230, Rationale for including Burn fields inside OrchardZSA Action Groups).

Action and burn encodings. An `OrchardZSAAction` occupies 372 bytes: `cv` (32), `nullifier` (32), `rk` (32), `cmx` (32), `ephemeralKey` (32), `encCiphertext` (132), and `outCiphertext` (80) (ZIP-230, OrchardZSA Action Group Description). The 132-byte ciphertext is the memo-bundle-era layout: the note plaintext gains the 32-byte asset base and replaces the in-band 512-byte memo with a 32-byte memo key K^{memo} (ZIP-231), the memo chunks themselves moving to a transaction-level memo bundle (fields `nMemoChunks`, `pruned`, `vMemoChunks`, present iff `fAllPruned` $= 0$) whose pruned entries are replaced by their digests (ZIP-230, Transaction Format; §6.3). The v6 note plaintext is

$$\texttt{leadByte} \,\|\, \text{LEBS2OSP}_{88}(d) \,\|\, \text{I2LEOSP}_{64}(v) \,\|\, \text{rseed} \,\|\, \text{asset\_base} \,\|\, K^{\text{memo}},$$

with $\text{asset\_base} = \text{LEBS2OSP}_{\ell_P}(\text{repr}_P(\text{AssetBase}))$, and the lead byte mandatory for all v6 note plaintexts (ZIP-230, note plaintext encoding). Upstream, the lead byte is the unassigned placeholder `{{LEADBYTE}}`: the original assignment `0x03` was taken by ZIP-2005, and no replacement is assigned (§6.6). For the same K^{memo} reason the Sapling `OutputDescriptionV6` shrinks `encCiphertext` to 100 bytes (ZIP-230, Sapling Output Description). An `AssetBurn` entry is 40 bytes: `AssetBase` (32, the encoding of $\text{AssetBase}^{\text{Orchard}}$) and `valueBurn` (`uint64`), consensus-checked non-zero and less than MAX_BURN_VALUE (ZIP-230, OrchardZSA Action Group Description; §6.4).

Issuance bundle. The issuance component carries five fields: `issuerLength` (`compactSize`), `issuer`, `nIssueActions`, `vIssueActions`, and `issueAuthSig`. The signature is present iff `nIssueActions` > 0; the first four fields are always present, and if `nIssueActions` $= 0$ then `issuerLength` MUST be 0 — an empty issuance bundle is exactly two zero `compactSize` values (ZIP-230, Transaction Format). An `IssueAction` carries `assetDescHash` (32 bytes), `nNotes` and `vNotes` at 115 bytes per `IssueNoteDescription`, and `flagsIssuance` with `finalize` in the LSB; `nNotes` may be zero, so a finalize-only action is expressible. An `IssueNoteDescription` is `recipient` (43 bytes, the raw Orchard payment-address encoding), `value` (`uint64`), `rho` (32), `rseed` (32). The `IssueAuthSignature` is `sizeSighashInfo` and `sighashInfo`, then `sizeSignature` and a signature "preceded by a `0x00` byte indicating a BIP 340 signature" (ZIP-230, issuance field encodings) — the same `0x00` scheme byte that prefixes the issuer key encoding (ZIP-227, Issuance Authorization Signature Scheme).

Versioned signatures. The `SaplingSignature` and `OrchardSignature` types prepend a `compactSize`-prefixed `sighashInfo` to the 64-byte RedJubjub or RedPallas signature (ZIP-230, signature encodings). ZIP-246 defines the versioning:

$$\text{sighashInfo} := [\,\text{sighashVersion}\,] \,\|\, \text{associatedData},$$

where version 0 is the "commit to all effecting data" algorithm; for v6, only version 0 is defined, with empty `associatedData`, for all four of Transparent, Sapling, Orchard, and Issuance (ZIP-246, Sighash versioning). The machinery buys the ability to introduce — and, critically, to retire — sighash algorithms per transaction version without a new format.

Coinbase. For coinbase transactions the `enableSpendsOrchard` and `enableZSAs` bits MUST be zero (ZIP-230, consensus rules), and ZIP-235 requires all coinbase transactions to be v6 from its activation, scheduled for NU7 (ZIP-230, Implications for Mining).

6.3 Digests: txid, sighash, and authorizing data (ZIP-246)

ZIP-246 extends the ZIP-244 digest tree — constructed for vanilla Orchard in the *Ironwood Guide*, §"Conservation without disclosure: value commitments and the binding signature" — from four top-level branches to six:

$$\text{txid} := \text{BLAKE2b-256}\big(\texttt{ZcashTxHash\_} \,\|\, \text{branchid},\ d_{\text{hdr}} \,\|\, d_{\text{transp}} \,\|\, d_{\text{Sap}} \,\|\, d_{\text{Orch}} \,\|\, d_{\text{issue}} \,\|\, d_{\text{memo}}\big),$$

the nodes T.1–T.6 of the ZIP, each itself a personalized BLAKE2b-256 (ZIP-246, TxId Digest). New relative to ZIP-244 are the fee and burn fields in the header digest, the Sapling output digests, the entire Orchard Action-Group subtree, the issuance subtree, and the memo subtree. The header digest T.1 is

$$d_{\text{hdr}} := \text{BLAKE2b-256}\big(\texttt{ZTxIdHeadersHash},\ \mathsf{version} \,\|\, \mathsf{versionGroupId} \,\|\, \mathsf{branchid} \,\|\, \mathsf{lockTime} \,\|\, \mathsf{expiryHeight} \,\|\, \text{LE}_{64}(\mathsf{fee}) \,\|\, \text{LE}_{64}(\mathsf{burnAmount})\big)$$

(ZIP-246, T.1). The Orchard branch T.4 hashes the concatenated per-group digests and the balance, $d_{\text{Orch}} = \text{BLAKE2b-256}(\texttt{ZTxIdOrchardHash}, d_{\text{AGs}} \,\|\, \text{LE}_{64}(v^{\text{Orchard}}_{\text{balance}}))$, with a defined empty case; each group digest (T.4a) covers a compact digest (nf, cmx, `ephemeralKey`, `encCiphertext`), a non-compact digest (cv, rk, `outCiphertext`), `flagsOrchard`, `anchorOrchard`, `nAGExpiryHeight`, and a burn digest over the (AssetBase, valueBurn) tuples (ZIP-246, T.4a). The issuance branch T.5 hashes `issuerLength` || `issuer` and a per-action digest (per action: `assetDescHash`, a per-note digest over (`recipient`, `value`, ρ, rseed), `flagsIssuance`), each level with a defined empty case (ZIP-246, T.5). The memo branch T.6 hashes a nonce and the concatenation of per-chunk digests; the v6 pruned memo bundle stores exactly these `memo_chunk_digest` / `memo_digest` values in place of pruned chunks, so pruning preserves the txid (ZIP-246, T.6; ZIP-230, Transaction Format). Table 5 collects the personalization strings; the implementation defines the same constants (§6.6).

Node	Personalization	Content
T.1	`ZTxIdHeadersHash`	header, incl. fee and burn amount
T.3b.i	`ZTxId6SOutC_Hash`	Sapling v6 outputs, compact
T.3b.ii	`ZTxId6SOutN_Hash`	Sapling v6 outputs, non-compact
T.4	`ZTxIdOrchardHash`	Action-Group digests, balance
T.4a	`ZTxIdOrcActGHash`	one Action Group
T.4a.i	`ZTxId6OActC_Hash`	nf, cmx, `ephemeralKey`, `encCiphertext`
T.4a.ii	`ZTxId6OActN_Hash`	cv, rk, `outCiphertext`
T.4a.vi	`ZTxIdOrcBurnHash`	burn tuples
T.5	`ZTxIdSAIssueHash`	issuer, issue-actions digest
—	`ZTxIdIssuActHash`	per action: desc. hash, notes, flags
—	`ZTxIdIAcNoteHash`	per note: recipient, value, ρ, rseed
T.6	`ZTxIdMemo___Hash`	nonce, chunks digest
—	`ZTxIdMemoCksHash`, `ZTxIdMemoCk_Hash`	memo chunk digests
A.1	`ZTxAuthTransHash`	scripts, now incl. per-input `sighashInfo`
A.3	`ZTxAuthOrchaHash`	group auth digests, `bindingSigOrchard`
A.3a	`ZTxAuthOrcAGHash`	`proofsOrchard`, spend-auth digest
A.3a.ii	`ZTxAuthOrSASHash`	`spendAuthSig` encodings
A.4	`ZTxAuthZSAOrHash`	`issueAuthSig` encoding

Table 5: ZIP-246 digest personalizations (ZIP-246, TxId Digest and Authorizing Data Commitment). Unnumbered rows are children whose labels the ZIP text assigns inconsistently; see the closing remark of this subsection.

Signature digest. The sighash reuses the tree with the transparent branch replaced by an input-specific variant:

$$\mathsf{sighash} := \text{BLAKE2b-256}\big(\texttt{ZcashTxHash\_} \,\|\, \mathsf{branchid},\ d_{\text{hdr}} \,\|\, d^{S}_{\text{transp}} \,\|\, d_{\text{Sap}} \,\|\, d_{\text{Orch}} \,\|\, d_{\text{issue}} \,\|\, d_{\text{memo}}\big),$$

produced per transparent input, per Sapling input, per OrchardZSA Action, and — new in v6 — per Issue Action; the Sapling, Orchard, issuance, and memo branches are identical to their txid-side counterparts, so the issuance bundle and the memo bundle enter every sighash (ZIP-246, Signature Digest). On this digest ZIP-227 hangs its authorization rule: the SigHash is the §4.10 SIGHASH transaction

hash with the ZIP-226 modifications, computed with `SIGHASH_ALL`, and the issuance authorization signature MUST satisfy $\mathsf{IssueAuthSig.Validate}(\mathsf{ik}, \mathsf{SigHash}, \texttt{issueAuthSig}) = 1$, where ik is decoded from the `issuer` field and its encoding is required to begin with `0x00`, the BIP-340 scheme byte (ZIP-227, Specification: Consensus Rule Changes).

Authorizing-data commitment. The commitment to authorizing data follows the same pattern: A.1 covers transparent scripts together with the new per-input `TransparentSighashInfo`; A.2 keeps the ZIP-244 structure with field encodings altered by `sighashInfo`; A.3 commits to the per-group proofs and spend-authorization signatures and to `bindingSigOrchard`; and a new A.4 commits to the `issueAuthSig` field encoding, with a defined empty case (ZIP-246, Authorizing Data Commitment).

Remark 6.1 (The reference text is itself unfinished)**.** ZIP-246 is internally incomplete even as withdrawn text: the authorizing-data section carries "TODO: Update the Authorizing Data Commitment below for the `sighashInfo` changes to the tx format," Rationale and Reference implementation are TBD, and the subtree labels are inconsistent — `issue_actions_digest` is listed as T.5c but headed T.5a with children T.5a.i, and the burn-digest children are labelled T.4b.i/T.4b.ii under node T.4a.vi (`zip-0246.rst`). Anyone respecifying the format inherits these defects as open editorial work; the implementation, not the ZIP, is currently the more precise artifact.

6.4 Transaction-level consensus rules

One Action Group. Although the format is a vector of Action Groups, ZIP-227 collapses it for OrchardZSA: per transaction, `nActionGroupsOrchard` MUST be 0 or 1, and `nAGExpiryHeight` MUST be 0 (ZIP-227, Specification: Consensus Rule Changes). The vector form is used by draft ZIP-228's proposed ZSA swaps, which would relax the one-group rule for swap bundles; it is not active in the core ZIP-227 rules.

Issuance requires an Action Group. A transaction with an issuance bundle MUST also contain at least one OrchardZSA Action Group (ZIP-227, Specification: Consensus Rule Changes). The rule is load-bearing twice over. First, issue-note ρ derivation consumes the first Action's nullifier: each issued note's ρ is computed by the function ZIP-227 names $\mathsf{DeriveIssuedRho}$ — constructed in §3.3 — keyed by $\mathsf{nf}_{0,0}$, the nullifier of the input note of the first Action in the first Action Group, over the Issue Action and note indices (ZIP-227, Computation of ρ), which gives every issued note a globally unique ρ because nullifiers are unique. Second, without a spend in the transaction the issuance-only SIGHASH would be replayable; the mandatory Action Group ties the issuance signature to a nullifier that consensus consumes (ZIP-227, Rationale).

Global issuance state. Nodes maintain a map $\mathsf{issued\_assets}$ from asset bases to triples $(\mathsf{balance}, \mathsf{final}, \mathsf{note\_ref})$, chained empty map → per-block → per-transaction. Burns decrement $\mathsf{balance}$, requiring $\mathsf{balance} \geq v$, *before* the transaction's issuance bundle is processed; issuance requires $\mathsf{final} \neq 1$, increments $\mathsf{balance}$ subject to the overflow bound $\mathsf{MAX\_ISSUE} = 2^{64} - 1$, and sets final when `finalize` = 1. The first issuance of an asset MUST include a *reference note*: value 0, paid to the default address of the all-zero Orchard spending key (ZIP-227, Global Issuance State and Reference Notes). The 43-byte raw encoding of that address is fixed in the ZIP and reproduced as a constant in the implementation (§6.6).

Burn rules and the extended balance equation. For every $(\mathsf{AssetBase}, v)$ in `assetBurn`: $\mathsf{AssetBase} \neq V^{\mathsf{Orchard}}$ (ZEC cannot be burned this way), $0 < v \leq \mathsf{MAX\_BURN\_VALUE}$, and no two entries share an AssetBase; the bound $\mathsf{MAX\_BURN\_VALUE} := 2^{63} - 1$ is chosen for compatibility with signed 64-bit value-balance fields (ZIP-226, Burn Mechanism). Burns enter value balance through the binding key. Where vanilla Orchard computes bvk by subtracting the committed declared balance from the sum of net value commitments (the *Ironwood Guide*, §"Conservation without

disclosure: value commitments and the binding signature"), ZIP-226 subtracts one zero-randomness commitment per burn entry as well:

$$\begin{aligned}\mathsf{bvk} := \Big(\sum_{i=1}^{n} \mathsf{cv}^{\mathsf{net},i}\Big) - \mathsf{ValueCommit}_0^{\mathsf{OrchardZSA}}\big(V^{\mathsf{Orchard}}, v^{\mathsf{Orchard}}_{\mathsf{balance}}\big) \\ - \sum_{(\mathsf{AssetBase},\, v) \in \mathtt{assetBurn}} \mathsf{ValueCommit}_0^{\mathsf{OrchardZSA}}(\mathsf{AssetBase}, v),\end{aligned}$$

and the prover signs the SIGHASH with $\mathsf{bsk} = \sum_{i=1}^{n} \mathsf{rcv}_i$ (ZIP-226, Burn Mechanism). The signature verifies only if every non-ZEC asset nets to exactly its declared burns — the same Pedersen-homomorphism argument as for ZEC, run per asset base.

Commitment-tree order. Note commitments enter the global tree in a fixed order: first, per Action Group, the commitments of the new OrchardZSA notes of each Action; then, per Issue Action, the issue-note commitments (ZIP-227, Specification: Consensus Rule Changes). Wallets and nodes must agree on this order to agree on note positions.

6.5 Fees (ZIP-317): specified in the fork, removed upstream

The fee treatment chains through three documents: ZIP-226 (Transaction Fees) defers to ZIP-317 "as described in ZIP 227"; ZIP-227 (Changes to ZIP 317) states that the conventional fee is altered for the presence of issuance actions and the creation of new Custom Assets, pointing at ZIP-317's Fee calculation section. Fees remain payable in ZEC only (ZIP-227, rationale). All variants share the conventional-fee form documented in the *Wallet Guide*, §"Fees (ZIP-317)":

$$\mathsf{conventionalFee} := \mathsf{marginalFee} \cdot \max(\mathsf{graceActions},\ \mathsf{logicalActions}),$$

with $\mathsf{marginalFee} = 5000$ zatoshis and $\mathsf{graceActions} = 2$ (ZIP-317, Fee calculation).

The pointer now dangles upstream: ZIP-317 (Last-Updated 2026-06-26) removed the ZSA contributions — its Change History records "Removed the ZSA issuance and asset-creation fee contributions (ZIP 227)" — leaving Revision 0 (active base), Revision 1 (Ironwood-pool Actions, NU6.3), and Revision 2 (memo bundles, draft) (ZIP-317, Revisions and Change History). The ZSA treatment survives in the QED-it fork copy (@ `fd71419`): a requirement that "users should be discouraged from issuing new 'garbage' Custom Assets," the parameter $\mathsf{creationCost} = 100$ logical actions, and two contributions summed into logicalActions alongside the base terms of the deployed rule:

$$\begin{aligned}\mathsf{contribution}_{\mathsf{ZSAIssuance}} &:= \mathsf{nZSAIssueNotes}, \\ \mathsf{contribution}_{\mathsf{ZSACreation}} &:= \mathsf{creationCost} \cdot \mathsf{nAssetCreations},\end{aligned}$$

where nZSAIssueNotes counts issue-note outputs and nAssetCreations counts Custom Assets newly added to the global issuance state (fork ZIP-317, Fee calculation). The rationale is state rent paid once: each new asset adds an issued_assets row tracked in perpetuity, while subsequent issuance, finalization, and burn only mutate the row. The implementation matches the fork text: `zcash_primitives`' `transaction/fees/zip317.rs` defines `CREATION_COST = 100` behind `cfg(zcash_unstable = "nu7")` and adds $\mathsf{nZSAIssueNotes} + 100 \cdot \mathsf{nAssetCreations}$ to the standard logical-action count. As with the deployed rule (the *Wallet Guide*, §"The deployed fee rule"), paying the conventional fee is not a consensus requirement in any ZIP-317 variant. Classification: the ZSA fee rule is *fork-specified*; whether it returns upstream as a ZIP-317 revision, as Ironwood's did, is undecided.

6.6 Implementation status at the pinned commits

Constants and serialization. All NU7 code in the librustzcash fork sits behind the configuration gate `cfg(zcash_unstable = "nu7")`. The constants are `V6_TX_VERSION = 6`,

V6_VERSION_GROUP_ID = 0x77777777, BranchId::Nu7 = 0x77190AD8, with NU7 activation None on Mainnet, 4,000,000 on Testnet, 1 on Regtest (zcash_protocol, constants.rs and consensus.rs). The v6 reader parses the header fragment, one vSighashInfo per transparent input (required to equal the version-0 encoding), the Sapling v6 bundle, the Orchard v6 bundle, and the issuance bundle (zcash_primitives, transaction/mod.rs). The Orchard reader rejects more than one Action Group ("A V6 transaction must contain at most one action group"), requires at least one Action, and rejects a nonzero nAGExpiryHeight; the writer emits exactly one group with nAGExpiryHeight = 0 (transaction/components/orchard.rs) — the ZIP-227 consensus rules hard-coded at the serialization layer. Issuance-bundle serialization matches ZIP-230, including the two-zero-compactSize empty bundle and rejection of the inconsistent issuer/action combinations (transaction/components/issuance.rs). The sighash implementation extends the v5 hash with the issuance digest for ZSA-capable versions, with sighash-info constants ORCHARD_SIGHASH_INFO_V0 = ISSUE_SIGHASH_INFO_V0 = [0x00] (sighash_v6.rs, sighash_versioning.rs). The orchard fork defines the ZIP-246 personalizations of Table 5 (bundle/commitments.rs, bundle/commitments/issuance.rs), asserting the 0x00 BIP-340 byte when hashing the 65-byte issuance signature encoding.

Verification code. Function derive_bvk_raw implements the extended balance equation of §6.4 verbatim (orchard fork, bundle.rs); validate_bundle_burn enforces the ZIP-226 burn rules — non-ZEC asset, non-zero amount, amount $\leq 2^{63} - 1$, no duplicates — plus presence in and sufficient supply under the global issuance state (bundle/burn_validation.rs); and verify_issue_bundle checks the BIP-340 signature over the 32-byte sighash, the per-note ρ against DeriveIssuedRho, non-finalization, u64-checked supply addition, and the reference note on first issuance, whose 43-byte RAW_REFERENCE_RECIPIENT constant equals the array in ZIP-227 (issuance.rs, constants/reference_keys.rs). The 0x84 domain separator for issued-note ρ lives in the QED-it zcash_spec fork (rev d5e8426). Proof sizes record the circuit cost: the ZSA aggregated proof has base size 2848 bytes and 2272 bytes per Action, against a vanilla base of 2720 (primitives/orchard_primitives_zsa.rs, orchard_primitives_vanilla.rs).

Divergences from the withdrawn ZIP text. Four are material; each is an explicit interim measure or a pending reassignment, not a bug, but each blocks byte-compatibility with ZIP-230/246 as written.

1. *No explicit fee field.* The v6 reader serializes no fee, and zip233Amount only behind the zip-233 cargo feature; the header txid digest correspondingly omits ZIP-246's T.1f fee field, with the comment "TODO: Factor this out ... when implementing ZIP 246 in full" (transaction/mod.rs, txid.rs).
2. *No memo bundles.* The forks implement the pre-ZIP-231 layout: OrchardZSA notes carry a full 512-byte memo, giving a 612-byte encCiphertext with an 84-byte compact prefix (52 vanilla bytes plus the 32-byte asset base), and Sapling v6 outputs are read in the v5 580-byte format (primitives/zcash_note_encryption_domain.rs, transaction/components/sapling.rs). No memo fields and no T.6 are implemented; instead the action-group txid digest inserts a non-spec memos digest (personalization ZTxIdOrcActMHash) between the compact and non-compact digests and hashes only the 84-byte compact slice into the compact digest, with TODOs "Remove once new Memo Bundles are implemented (ZIP-231)" (primitives/orchard_primitives_zsa.rs).
3. *Lead byte.* The implementation hard-codes NOTE_VERSION_BYTE_V3 = 0x03; upstream ZIP-230 withdrew that claim after ZIP-2005 took 0x03, leaving the ZSA lead byte an unassigned placeholder. The fork's choice predates upstream ZIP-2005 assigning 0x03 to Ironwood notes and directing future proposals to pick a new value; the collision is fork-only context, with no

format ambiguity on Zcash mainnet. Reassignment is a hard prerequisite for any deployment: the lead byte selects the note-plaintext parser.

4. *Split-note* ψ^{nf}. Upstream ZIP-226 now derives the split-note nullifier randomizer as $\psi^{\mathsf{nf}} = \mathrm{ToBase}\big(\mathsf{PRFexpand}_{\mathsf{rseed}_{\mathsf{nf}}}(\texttt{0x0A} \,\|\, \rho^{\mathsf{old}})\big)$ with $\mathsf{rseed}_{\mathsf{nf}}$ sampled uniformly at random, the first-byte domain `0x0A` "reserved by ZIP 2005 for this use" (ZIP-226, split-note nullifier); the fork at `cf801a5d` realizes the older fork text (uniformly random ψ^{nf}) via a separate `rseed_split_note` under the ordinary ψ domain `0x09` (`note.rs`, `circuit.rs`).

Audit scope. Public reporting supports both standing claims, at scope: Least Authority's *OrchardZSA Protocol Security Audit Report* (ZCG-commissioned, Updated Final 30 January 2025) reviewed the QED-it `orchard` `zsa1`-vs-`main` diff (PR QED-it/orchard#7, the ZSA circuit extensions) and the QED-it `halo2` changes (PR QED-it/halo2#18) at revisions `a7c02d22/01e85a5f` and identified no issues; ZecHub *Shielded News* Vol. 61 (2025-12-17) reports a feature-complete ZSA build on QED-it's public Zebra testnet; both sources are pinned in §1.2. Neither covers `librustzcash-zsa` or the v6 carrier layer, and the audited revisions predate `cf801a5d`. Two facts bound what the past audit covers: the audited revisions predate both the NU6.2 Orchard circuit remediation (ZIP-257) — and the ZSA circuit is a fork of the Orchard circuit — and the ZIP-2005-relative rebasing of upstream ZIP-226. A mainnet path implies re-audit against both.

6.7 Wallet integration

ZIP-230 and ZIP-226 impose two obligations. First, all wallets SHOULD switch to sending only v6 transactions once permitted: a v5 transaction reveals that all its Orchard notes are ZEC, and only v6 OrchardZSA outputs are quantum-recoverable (ZIP-230, Implications for Wallets; ZIP-226, Backwards Compatibility with ZEC Notes). Second, wallets MUST support parsing and trial-decrypting v6 outputs — the new lead byte and the memo-bundle changes — by activation, or rescan once support lands: funds sent to an un-upgraded wallet are temporarily inaccessible, because the new lead byte selects different rcm/ψ derivations and a v5-decryption wallet rejects the note (ZIP-230, Implications for Wallets). Payment requests have a specified extension path: ZIP-321's `req-asset` parameter carries an Asset Identifier and amount (the *Wallet Guide*, §"Custom assets: `req-asset` (specified, not deployed)").

Multi-party and hardware signing is the open end. No PCZT structure exists for OrchardZSA bundles or issuance bundles: the librustzcash fork's transaction extractor panics with "PCZT support for ZSA is not implemented" when handed an OrchardZSA bundle, the orchard fork's PCZT module is written against the vanilla flavor only, and the draft PCZT v2 targets ZIP-229's Ironwood v6, not ZSA (the *Wallet Guide*, §"Partially created transactions (PCZT)" and §"PCZT v2: deferred anchors and the Ironwood bundle"). Classification: *open* — hardware-wallet and multi-party signing for ZSAs is unspecified in any draft.

6.8 Outlook

ZIP-226 and ZIP-227 remain Status: Draft. ZIP-226's Deployment section reads "The Zcash Shielded Assets protocol is scheduled to be deployed in Network Upgrade 7 (NU7). This ZIP currently assumes that this will be the case"; ZIP-227's Deployment is TBD and its test-vector and reference-implementation entries are "LINK TBD," with ZIP-226 naming the QED-it forks and `QED-it/zcash-test-vectors` as reference artifacts (ZIP-226/227, Deployment). The deployment vehicle is thinner still: the NU7 deployment draft is unnumbered (`draft-arya-deploy-nu7`, Status: Draft, replacing the withdrawn ZIP-254), fixes `CONSENSUS_BRANCH_ID` = `0x77190AD8` and minimum protocol versions (170150 Testnet, 170160 Mainnet) with activation heights TBD, and lists *no* ZIPs. Those version numbers now conflict with ZIP-204's NU7 assignment of 170180 and 170190, so the deployment draft needs an editorial update. The repository's NU7 candidate list (ZIPs 218, 230, 231, 233, 234, 235, 2002, 2003, plus a possible ZIP-317 update) does not include ZIP-226/227, still

names the withdrawn ZIP-230, and states that "no decision has been made on whether to include each of these ZIPs" (`zcash/zips` README, NU7 Candidate ZIPs). Governance sentiment is split: in the February 2026 NU7 sentiment polls, ZCAP supported ZSAs at 70.4% while the coinholder poll ran 98.6% against; the Zcash Foundation did not recommend ZSAs for NU7, writing that "before any decision is made, we need to understand why coinholders oppose ZSAs so strongly" (zfnd.org, "NU7 Polling Results: What We Heard and Where We Go From Here", 2026-02-23, `https://zfnd.org/nu7-polling-results-what-we-heard-and-where-we-go-from-here/`).

Between the pinned implementation and any deployment stand three respecification problems, none of which is per-object protocol work.

1. *A carrier.* Transaction version 6 now belongs to NU6.3's Ironwood format (ZIP-229), whose non-requirements exclude the ZSA machinery (§6.1). The designated successor for what ZIP-230/246 deferred — an extensible transaction format — is ZIP-248, which exists only as open PR `zcash/zips#1156`. The entire ZSA and memo-bundle format stack therefore has no merged specification; if ZSAs are selected for NU7, the byte format of §6.2 must be respecified under new numbers, and §6.3 re-anchored to whatever digest ZIP results.
2. *A base protocol.* Upstream ZIP-226 is now "defined relative to the Zcash protocol with the changes specified in ZIP 2005 applied," with ZIP-2005 (Ironwood Quantum Recoverability) "expected to deploy before any ZSA activation" (ZIP-226, header notes). NU6.3 (ZIP-258, testnet activation 4,134,000, Mainnet 3,428,143) introduces the Ironwood pool and restricts transfers into the Orchard pool (ZIP-2006, currently a Reserved stub). No ZIP specifies whether OrchardZSA then targets the restricted Orchard pool or is rebased onto Ironwood; the audited revisions predate the entire NU6.2/NU6.3 pivot. Classification: *open*.
3. *Reconvergence of spec and code.* The divergence list of §6.6 — fee field, memo bundles, lead byte, ψ^{nf} — plus the orphaned fee treatment of §6.5 must land on one side or the other: either the respecified ZIPs adopt what the code does, or the code moves. Every item is known and mechanical; none is done.

The summary judgment this volume can defend: the ZSA transaction layer is *designed and implemented, but currently unspecified* in the sense this series requires — there is no merged normative text a mainnet node could be held to. The byte-level facts of this section are fixed by withdrawn ZIP text and pinned fork code, and they will remain the diff base for the respecification; the open items are enumerated above and in the scope section, and every one of them is legible, which is more than can be said for most protocols at this stage.